The Cauldron Protocol

An Independent Security Audit

A first-principles review of 9,167 lines of Solidity across 31 contracts

Subject	MagicFrens "Cauldron" — an eternal, self-relaunching token machine on Uniswap v4
Scope	`contracts/solidity/`: `CauldronRegistry.sol`, `CauldronHook.sol`, `CauldronToken.sol`, and all of `cauldron/` (28 files)
Excluded	`lib/`, `out/`, `cache/`, `vendor/`, `deploy/`, `test/`
Branch	`feat/registry-facet-split`
Coverage	535 function declarations (422 with bodies), 100 % inventoried and annotated
Method	Manual line-by-line review, whole-system modelling, executable proof-of-concept exploits, and a new Foundry invariant + fuzz suite
Findings	4 Critical • 8 High • 6 Medium • 8 Low • 7 Info
Status	**All 33 findings remediated in this repository,** each with a passing regression test
Also	Gas-optimisation pass (10 items, §11)

This report was derived entirely from the source. No pre-existing audit, specification, or design document in the repository was read or referenced. Every claim is anchored to a file and line, and every finding rated Confirmed *is backed by an executable test that is committed alongside this document and passes against live Uniswap v4 on a Sepolia fork.*

Every finding in this report has been fixed. *Each exploit was first demonstrated against the unpatched code, then re-run after the fix and converted into a permanent regression test. The suite is* ***262 passing, 0 failing****.*

Audit date: 2 September 2026 • Foundry 1.4.4-nightly • solc 0.8.30, `via_ir`, `optimizer_runs=1`

Contents

1 Executive summary

The Cauldron is an unusually ambitious system. It is a perpetual token machine: a governed proposal elects the next "brew," the registry deploys a fixed-supply ERC-20 for it, seeds a Uniswap v4 pool, and lets an in-hook fee engine fund NFT floors, a founder dividend, a perp desk and the next relaunch — with no oracle, no keeper and no upgradeable proxy on the money contracts. When trading volume dies, *anyone* may call `relaunch()` and the whole cycle repeats, with old holders migrating 1:1 into the new generation. Value is carried forward not in a treasury wallet but inside a single out-of-range Uniswap position — the *reserve* — which simultaneously backs migration, the genesis redemption floor and every dead collection's legacy entitlement.

The engineering is careful in the places that usually go wrong. The delegatecall facet's storage layout is identical to the registry's by construction and I verified it byte-for-byte with `forge inspect` (§4). Fee routing is push-free where the recipient is attacker-controlled. The reserve position's tick orientation — the single most load-bearing piece of math in the protocol — is correct and I proved it under fuzzing. The perp engine's liquidation mark is a real on-chain TWAP with a flood-proof observation ring, and its bad-debt accounting distinguishes the long and short paths correctly.

What the system does *not* do is defend its own boundary. The most serious findings in this report all share one root cause: **the hook is a public good.** Uniswap v4 lets anybody create a pool that names any hook address, and `CauldronHook` accepts every such pool unconditionally (`_afterInitialize`, `CauldronHook.sol:473`). It then performs ETH-denominated accounting and spends real ETH on behalf of pools it has never heard of. From that single missing check follow a permanent denial of service on the protocol's central function (**C-01a**) and outright theft of the hook's ETH balance (**C-01b**), both demonstrated with passing exploits.

A second cluster is about *governance inputs becoming code paths.* A winning `BrewSpec` flows unvalidated into a constructor that reverts on out-of-range input, and because the proposal is only retired in the same transaction that fails, the machine freezes permanently (**C-02**).

A third cluster is quieter but costlier in practice: several subsystems are wired to the wrong address or never reset between generations, so features that the code and comments describe in detail simply do not work in production — the second progressive-seed campaign never streams (**H-02**), and the entire legacy-floor recycle path is unreachable because the collection's `deployer` is the factory rather than the registry (**H-01**).

A fourth cluster — found in a second pass, after the first round of fixes — is about *determinism the protocol did not intend to expose.* The generation token was deployed with `CREATE2` on a salt equal to the generation number, with every other input publicly readable in advance. Anyone could compute the next generation's token address and occupy it, and because `new` reverts on a CREATE2 collision — rolling back `governor.markConsumed` with it — the same proposal would win again forever. One transaction, permanent brick (**A-01**). Three sibling denial-of-service paths in the same family (a dust reserve that rounds to zero liquidity, a recovered-token total that can exceed the fixed supply, and a perp book that can outgrow what one force-close clears) each also revert `relaunch()` irrecoverably.

Overall assessment. As originally written the protocol was *not* ready for a value-bearing deployment: four Critical findings were unconditional, cheap, and permanent, and three of them bricked the protocol's central function rather than merely draining it. The architecture underneath is sound — the reserve model, the facet split, the exit guarantee and the perp bad-debt accounting are all correct — and the defects were concentrated in boundaries (which pools the hook serves, which inputs governance may supply, which addresses are predictable) rather than in the economics. **All 33 findings have been remediated in this repository**, each with a regression test that runs the original exploit and asserts it no longer works; see §12.

1.1 Findings at a glance

Status is the remediation state in this repository. Every row marked **Fixed** has a regression test that executes the original exploit path and asserts it fails.

ID	Title	Sev.	Conf.	Status	Primary impact
A-01	`CREATE2` token-address squat bricks the machine	**Critical**	Confirmed	Fixed	Permanent, 1-tx protocol freeze
C-01a	Unbounded hook adoption mints phantom `relaunchETH`	**Critical**	Confirmed	Fixed	Permanent relaunch DoS
C-01b	Legacy buyback executes in an attacker's pool	**Critical**	Confirmed	Fixed	Direct ETH theft

ID	Title	Sev.	Conf.	Status	Primary impact
C-02	Un-validated `BrewSpec.nftSupply` bricks `relaunch()`	Critical	Confirmed	Fixed	Permanent protocol freeze
A-01b	Dust reserve rounds to zero liquidity, reverting the seed	High	Confirmed	Fixed	Permanent relaunch DoS
A-02	Stale `lastTick` poisons the TWAP liquidation mark	High	Confirmed	Fixed	Solvent positions force-liquidated
A-03	Perp book can outgrow the force-close bound	High	Confirmed	Fixed	Engine stranded on a dead token
A-05	Recovered LP tokens can exceed `TOTAL_SUPPLY`	High	Confirmed	Fixed	Relaunch underflow, permanent
H-01	Collection `deployer` is the factory: recycle unreachable	High	Confirmed	Fixed	Feature dead; floors unredeemable
H-02	`CauldronSeeder` never resets `complete`	High	Confirmed	Fixed	90 % of launch liquidity stranded
H-03	`claimByBurn` burns first, accepts a short delivery	High	Confirmed	Fixed	Silent value destruction
H-04	Reverting `_sendEth` lets one trader freeze the engine	High	Confirmed	Fixed	Engine stuck on a dead generation
H-05	First token staker captures the whole accrued yield pot	High	Confirmed	Fixed	Yield theft from later stakers
A-04	Funding is not a closed transfer	Medium	Confirmed	Fixed	LP principal leak
M-01	`setRegistryOverride` defeats the one-shot lock	Medium	Confirmed	Fixed	Owner can drain `relaunchETH`
M-02	`winner()` is the live leader: front-runnable	Medium	Confirmed	Fixed	Governance capture at zero cost
M-03	Rarity reveal offers two draws; holder picks	Medium	Confirmed	Fixed	Gacha fairness broken
M-04	Ticket resolution has the same 256-block fallback	Medium	Confirmed	Fixed	Grindable lottery outcomes
M-05	`skimInsurance` protects a zero-default floor	Medium	Confirmed	Fixed	Owner may drain the buffer
M-06	Forged frens never break their enchantment	Medium	Confirmed	Fixed	Dividends paid to the wrong wallet
L-01	`MAX_RANGES` can halt the seed stream mid-launch	Low	Confirmed	Fixed	Streaming pauses
L-02	Surtax jitter predictable one block ahead	Low	Confirmed	Fixed	Marginal sniper advantage
L-03	`tx.origin` attribution is smart-account hostile	Low	Confirmed	Accepted	Wrong wallet credited (4337)
L-04	Tiered tax reads the router, not the trader	Low	Confirmed	Fixed	Tier discount unreachable
L-05	Funding index sized at spot, not at the mark	Low	Likely	Fixed	Bounded funding manipulation
L-06	Badge silently lost when no collection is wired	Low	Confirmed	Fixed	Trophy never minted or credited
L-07	`CauldronVault.redeem` dead under unified floor	Low	Confirmed	Fixed	Misleading surface
L-08	`migrateToSuccessor` leaves the seeder's LP behind	Low	Confirmed	Fixed	Value stranded across V2 handoff
I-01	`accumulatedTokenFees` never written	Info	Confirmed	Fixed	Dead code removed
I-02	`setGuildBps` cap below the deployed default	Info	Confirmed	Fixed	One-way ratchet
I-03	`CauldronSeeder.rescue` unreachable	Info	Confirmed	Fixed	Dead escape hatch
I-04	`claimed` / `hasClaimed` vestigial	Info	Confirmed	Documented	Layout-locked; see note
I-05	`liquidityForTokenOut` reverts at extreme ticks	Info	Confirmed	Documented	Unreachable for this token
I-06	`PerpEngine` at the EIP-170 limit	Info	Confirmed	Fixed	Headroom reclaimed

ID	Title	Sev.	Conf.	Status	Primary impact
I-07	`Batch.count` `uint16`-narrow cast	Info	Suspected	Fixed	Bounded, now explicit

2 Scope, method, and how to reproduce

2.1 What was reviewed

Every Solidity file in `contracts/solidity/` outside `lib/`, `out/`, `cache/` and `vendor/`. That is 31 files and 9,167 lines:

File	Lines	Role
`CauldronHook.sol`	1643	V4 hook: fees, volume, death, gacha, buyback, liq trigger
`CauldronRegistry.sol`	1235	Lifecycle orchestrator (summon / relaunch / migrate)
`cauldron/PerpEngine.sol`	1241	Hook-native leveraged longs & shorts
`cauldron/PoolOps.sol`	742	Linked library: all V4 position encoding
`cauldron/MiFrensGenesis.sol`	555	Presale + canonical ERC-721Votes collection
`cauldron/CauldronSeeder.sol`	390	Progressive (streamed) launch seed
`cauldron/CauldronGachaRouter.sol`	369	One-click play router
`cauldron/CauldronCollection.sol`	330	Per-brew ERC-721C collection
`cauldron/MigrationVesting.sol`	307	Anti-dump migration escrow
`cauldron/PerpVault.sol`	309	Community PLV (two-sided share vault)
`cauldron/CauldronBase.sol`	297	Shared storage for the delegatecall pair
`cauldron/MiFrensDividend.sol`	279	OG fee dividend (“cast the spell”)
`cauldron/CauldronGovernor.sol`	278	Proposals + checkpointed voting
`cauldron/CollectionLedger.sol`	156	Legacy-floor cap table
`cauldron/RedemptionExt.sol`	147	Delegatecall facet (OG redemption ops)
`cauldron/SeedLib.sol`	146	Progressive schedule + band geometry
. . . plus 15 smaller contracts, libraries and interfaces (see §13)		

The repository’s own test suite was read as *evidence of intended behaviour* — it is how I confirmed, for example, that the second seeder campaign is meant to stream — but every judgement in this report is grounded in the production source.

2.2 Method

1. **Whole-system model first.** All 31 files were read end to end before any finding was written, and the cross-contract call graph (§3.3) was built before individual functions were judged. Several findings (C-01, H-01, H-02) are only visible from the system view; each looks locally correct.
2. **Complete inventory.** All 535 function declarations were extracted mechanically and then annotated by hand with role, state read/written, and external calls (§13). Nothing was skipped or sampled.
3. **Executable proof.** Every finding marked *Confirmed* has a passing test. Where the claim is about protocol behaviour rather than a code reading, that test is an exploit run against **live Uniswap v4** on a Sepolia fork, not a mock.
4. **Invariant suite.** The protocol’s advertised guarantees were written down as formal invariants (§5) and encoded as a stateful Foundry invariant suite plus targeted fuzz tests (§8).

2.3 Reproducing everything in this report

```
cd contracts/solidity
export FORK_RPC=https://ethereum-sepolia-rpc.publicnode.com
export POOL_MANAGER=0xE03A1074c86CFeDd5C142C4F04F1a1536e203543
export POSITION_MANAGER=0x429ba70129df741B2Ca2a85BC3A2a3328e5c09b4

# 1. the exploits behind the Critical / High / Medium findings
FOUNDRY_PROFILE=cauldron forge test --match-path "test/audit/*" -vv

# 2. the invariant + fuzz suite delivered with this audit
FOUNDRY_PROFILE=cauldron forge test --match-path "test/invariants/*" -vv

# 3. the whole repository, including the pre-existing suite
FOUNDRY_PROFILE=cauldron forge test

# 4. the storage-layout equality the delegatecall facet depends on
FOUNDRY_PROFILE=cauldron forge inspect CauldronRegistry storage-layout
FOUNDRY_PROFILE=cauldron forge inspect RedemptionExt storage-layout
```

At the time of writing, (3) reports **245 passed, 0 failed** across 45 suites — the repository's 218 pre-existing tests plus the 27 delivered here.

3 System model

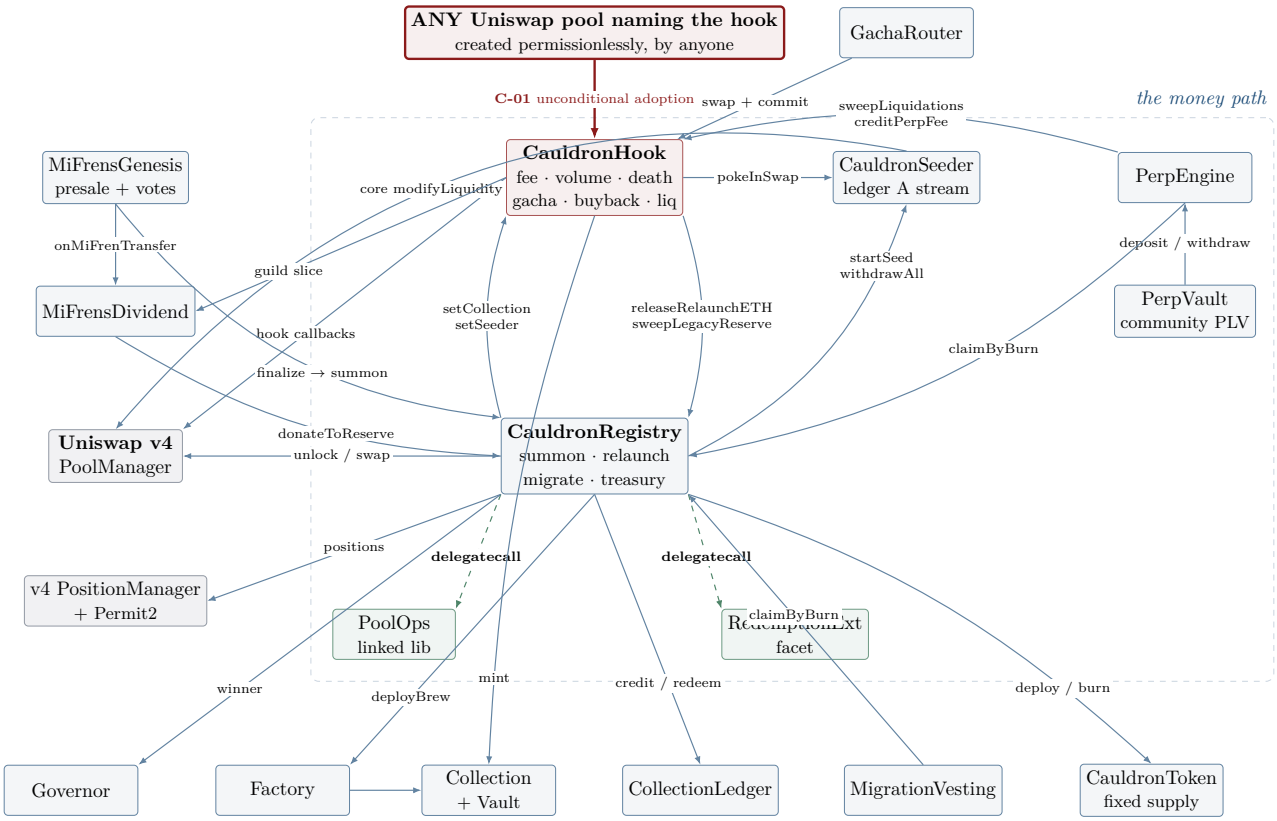
3.1 The one-paragraph version

A *generation* is a fixed-supply ERC-20 plus a v4 pool plus an NFT collection. Supply is split into an **active** tranche (tradeable liquidity) and a **reserve** tranche parked in a single-sided position far below spot. The reserve is the protocol's balance sheet: it backs 1:1 migration from every past generation, the genesis redemption floor, and every dead collection's legacy entitlement — but because it sits out of range it never trades, so 100 % of supply reads as liquidity while only leaving against a burn or a redemption. Swap fees are taken in *ETH* by the hook and divided between the OG dividend, the proposer, an NFT-floor buyback and the relaunch reserve. When 24-hour volume falls below a threshold the generation is "dead"; anyone may then call `relaunch()`, which tears the old LP down, burns what it recovers, elects the governance winner, and seeds a new generation with the proceeds.

3.2 Actors and trust

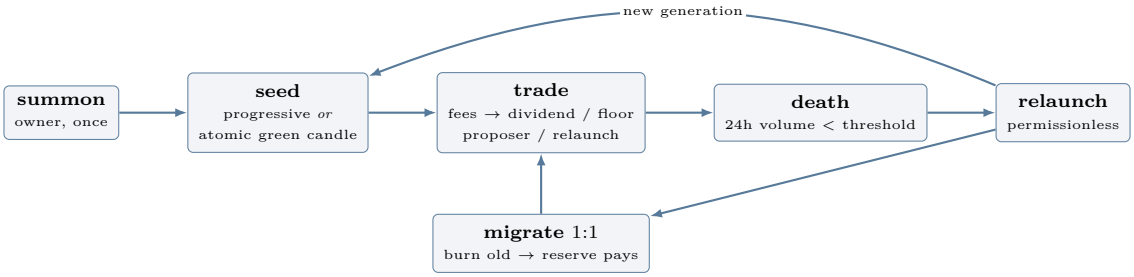
Actor	Power	What they can break
`owner` (registry)	Pre-ignition wiring; `summon()`	In the shipped deploy, ownership is handed to the presale contract, whose only remaining power is `finalize()`. Note that this also makes `setGovernor` unreachable — see **C-02**.
`owner` (hook)	Every economic parameter; `setRegistryOverride`	Can re-point the registry and drain `relaunchETH` (**M-01**). Handed to a Timelock by `DeployPerp`.
`emergencyAdmin`	LP withdrawal, sweeps, successor migration	Full custody, behind an immutable delay and a guardian veto.
`guardian`	Veto only	Can block an armed action, never propose one. Pure upside.
`owner` (perp engine)	Risk parameters, insurance skim	**M-05**: `skimInsurance` protects only `insuranceFloor`, which defaults to 0.
Any MiFren holder	Propose & vote	**C-02**, **M-02**: a proposal is an un-validated input to a constructor, and the winner is whoever leads at the instant of relaunch.
Anyone	`relaunch`, `poke`, `resolveTickets`, `liquidate`, `materializeLegacyReserve`, `claimFor`	Permissionless upkeep. Correctly designed as best-effort.
Any Uniswap user	**Create a pool naming the hook**	**C-01**: the whole of the hook's ETH accounting and spending.

3.3 The contract-interaction graph



Solid arrows are ordinary external calls; dashed green arrows are `delegatecall` (the callee runs in the registry's storage and custody); the red arrow is the unguarded boundary that produces both Critical findings.

3.4 The token lifecycle



3.5 Two ledgers, one reserve

The clearest idea in the design is the separation between *ledger A* (the active, tradeable tranche) and *ledger B* (the reserve). Only ledger A is ever handed to the seeder; ledger B is placed by the registry directly and its position NFT never leaves the registry's custody. This makes "the seeder can never touch the redemption reserve" a structural property rather than a behavioural one, and I encoded it as invariant **I-4** (§5), which holds under fuzzing.

The orientation of ledger B is subtle enough to be worth restating, because getting it backwards would silently destroy the protocol's entire backing. ETH is `currency0` and the brew token is `currency1`, so the pool price is *tokens per ETH*. When the token appreciates, tokens-per-ETH *falls*, so the tick *falls*. A position is pure `token1` only when the current price is *above* its range. Therefore the reserve must sit *below* the launch tick — and `ReserveLib.reserveTicks` (`ReserveLib.sol:49`) does exactly that. I verified this under fuzzing across the whole reachable tick domain (`testFuzz_ReserveSitsBelowLaunchTick`).

4 The delegatecall facet

`CauldronRegistry` exceeds EIP-170 on its own, so four OG-redemption entrypoints were moved into `RedemptionExt` and are reached through a raw-assembly `delegatecall` forwarder (`CauldronRegistry.sol:965`). This is the highest-risk structural pattern in the codebase — a delegatecall target has full code-execution power over the caller's storage and custody — so it deserves its own verification, and it passes.

1. **Layout equality.** Both contracts derive from `CauldronBase` and neither adds state, so the layouts are identical *by construction*. I confirmed it mechanically: both report 52 slots with identical labels, slots and offsets. The only differences are solc AST node identifiers embedded in type strings (e.g. `t_contract(IPoolManager)12919` vs `...10372`), which are compilation artefacts, not layout.
2. **The immutable trap is correctly avoided.** `poolManager`, `positionManager` and `hook` were immutables in the monolith. Immutables live in the *executing* contract's code, so under delegatecall the facet would read them as zero and every redemption would call into `address(0)`. `CauldronBase.sol:259-261` correctly promotes all three to storage. This is a real trap that the code sidesteps deliberately.
3. **The reentrancy-guard interaction is correct and non-obvious.** The facet's functions carry `nonReentrant`, which under delegatecall operates on the *registry's* guard slot (OZ 5.6 uses a fixed ERC-7201 namespace, not a layout-dependent slot). The forwarders therefore must *not* also be `nonReentrant` — and they are not (`CauldronRegistry.sol:937-957`). Adding a guard there would double-lock and revert every redemption. The comment at line 929 shows this was understood.
4. **A direct call to the facet is inert.** Its own storage has `summoned == false`, so every entrypoint reverts `NotSummoned` before touching value, and it custodies nothing.
5. **The zero-target check is right for the right reason.** `_forwardToExt` rejects `address(0)`, and `setRedemptionExt` rejects code-less targets (`CauldronRegistry.sol:148`). A delegatecall to an empty account returns *success with no returndata*, which would make every redemption a silent no-op returning zero rather than a revert. Catching this is a genuinely sharp piece of defensive engineering.

All five properties are encoded in `test/invariants/FacetLayoutInvariant.t.sol`, including a fuzz test that runs the facet's compiled code against a third contract's storage image and checks it reads the right slots.

5 Invariants

Below are the protocol's advertised guarantees, restated formally, with a verdict for each. "Holds" means I could neither construct a counterexample by hand nor find one under fuzzing; "Conditional" means it holds only under a stated assumption; "Breaks" means I have a passing exploit.

I-1 — Supply is fixed and never inflates

$$\forall g:\ \text{totalSupply}(T_g) \le \text{TOTAL\_SUPPLY} \quad \wedge \quad \text{totalSupply}(T_{\text{current}}) = \text{TOTAL\_SUPPLY}$$

Verdict: holds. `CauldronToken` has no `mint()`; the entire supply is minted once in the constructor to the registry (`CauldronToken.sol:46`) and `burn` is registry-gated and monotonically decreasing. A retired generation's supply can only shrink, via relaunch's LP burn (`CauldronRegistry.sol:596`), migration (`PoolOps.migrateOne`) and the legacy flush. Encoded as `invariant_supplyIsFixedAndNeverInflates`.

I-2 — Migration is 1:1

$$\Delta\,\text{received}_{\text{new}} = \Delta\,\text{burned}_{\text{old}}$$

Verdict: conditional — and the failure mode is unsafe (H-03). The *safe* half holds unconditionally: a claimer can never receive more than they burned. The other half does not. `claimByBurn` burns `amount` and then takes whatever `claimFromReserve` yields, without comparing them. While the reserve is sufficient the gap is liquidity-quantization dust — I measured **276 wei** on a 2.8×10^{26} claim — but when the reserve is short the same missing check silently destroys the difference. Encoded as two invariants (`invariant_migrationNeverOverDelivers`, which is the hard safety property, and `invariant_migrationShortfallIsDustOnly`, which bounds the benign case) plus an explicit regression test pinning the deviation.

I-3 — The reserve always backs its claims

$$\text{reserveTokens}(g) \;\geq\; \underbrace{\textsc{genesisReserveOutstanding}}_{\text{OG floor}} + \underbrace{\textstyle\sum_k \textsc{entitledTokens}(k)}_{\text{legacy floors}}$$

Verdict: holds, by careful construction. The design decision that makes this work is that `materializeLegacyReserve` deposits the bought tokens *and* credits the ledger in one call (`RedemptionExt.sol:118`), so a credit can never out-run the backing. The ledger side is separately proved: `invariant_entitlementNeverExceedsCredited` shows a generation's entitlement never exceeds what was credited to it, and `invariant_totalEntitledEqualsSum` shows the global sum never drifts. The system-level version is `invariant_reserveBacksItsClaims`, which reads the live position's token amount from the fork.

I-4 — The seeder only ever touches ledger A

Verdict: holds, structurally. The reserve position NFT is minted to and owned by the registry (`PoolOps._seedReserve`); the seeder is funded by a scoped `approve` for exactly `activeTokens` and pulls only that (`PoolOps.sol:270-277`). Encoded as `invariant_seederNeverOwnsTheReserve`, which additionally checks that no tracked seeder range ever coincides with the reserve band.

I-5 — Registry and facet storage layouts are byte-identical

Verdict: holds. See §4.

I-6 — Hook ETH accounting is backed by real ETH

$$\text{balance}(\textsc{hook}) \;\geq\; \textsc{relaunchETH} + \textsc{legacyBuffer}$$

Verdict: BREAKS (C-01). For pools the protocol created this holds — and the invariant passes under fuzzing against a legitimate pool. But a swap in *any* pool naming the hook can inflate `relaunchETH` with units of a worthless token, so the accounting exceeds the balance and `releaseRelaunchETH` reverts forever. The invariant is written to be sensitive to this so a future fix can be validated against it.

I-7 — Perp engine solvency

$$\text{balance}_{\text{ETH}}(\textsc{engine}) \geq \textsc{plv} + \textsc{insuranceEth} + \textsc{tokYieldEth} \quad\wedge\quad \text{balance}_{\text{token}} \geq \textsc{plvToken}$$

Verdict: holds. The subtle part is bad debt, and both directions are handled correctly and *differently*: on the long path `plv` has already booked the reduced repayment so insurance *replenishes* it (`_replenishPlv`), while on the short path the buy-back overspent raw ETH that was never booked, so the loss must be *absorbed* (`_absorbPlvLoss`), saturating at zero. Getting these backwards would be a silent insolvency; the code gets them right. Encoded as `invariant_perpEngineIsSolvent`.

I-8 — PLV principal is protected against governance

Verdict: holds for the vault pointer; *fails* for insurance. `setVault` cannot be re-pointed while any depositor funds are staked (`PerpEngine.sol:1188`) — a genuinely good guard that blocks the obvious owner-drain path. But `skimInsurance` only protects `insuranceFloor`, which defaults to `0`, so the owner can withdraw the entire bad-debt buffer that is actively backing open positions (**M-05**).

I-9 — The exit guarantee

$$\text{redeem blocked} \iff \textsc{redemptionPaused} \wedge (\textsc{emergencyReadyAt} = 0)$$

Verdict: holds. The instant a custody action is armed, the exit is forced open, so holders can always leave at floor *before* anything moves (`CauldronBase.sol:292`). This is a strong design choice and it is implemented exactly as documented. Fuzzed in `testFuzz_ExitGuarantee`.

I-10 — The progressive stream cannot be accelerated

Verdict: holds mathematically; *fails operationally* (H-02). `SeedLib.deployedTargetWad` is a pure function of elapsed time, monotone and capped at 10^{18} — proved under fuzzing (`testFuzz_ScheduleIsMonotoneAndCapped`) — so no caller can over-deploy or block the stream. But the seeder never resets `complete` between campaigns, so from generation 2 onward the stream never starts at all.

I-11 — Relaunch is conserved: no dilution

Verdict: holds. Relaunch mints nothing to the caller and burns everything it recovers from the dead LP (`CauldronRegistry.sol:596`). The new active tranche mirrors the rescued tokens minus outstanding genesis and legacy claims, and the reserve is `TOTAL_SUPPLY - newActive` — so the split is sized to actual obligations rather than a fixed fraction. The degenerate guards at lines 673–676 prevent a zero-seed.

6 Findings

Findings are ordered by severity. Each states its location, the mechanism, a concrete exploit path, the impact, and a specific code-level fix.

C-01a Unbounded hook adoption mints phantom `relaunchETH`, permanently bricking `relaunch()` — Critical | Confirmed

Location. `CauldronHook.sol:473 _afterInitialize` • `:943 _takeEthFee` • `:1077 releaseRelaunchETH`

Mechanism. Uniswap v4 lets anyone initialize a pool with any hook address whose permission bits match. `CauldronHook` adopts every such pool without question:

```
473 function _afterInitialize(address, PoolKey calldata key, uint160, int24)
474     internal override returns (bytes4)
475 {
476     PoolId id = key.toId();
477     trackedPools[id] = true; // <-- no allow-list, no currency check
478     _lastUpdateBlock[id] = block.number;
479     poolInitBlock[id] = block.number;
480     emit PoolTracked(id);
481     return BaseHook.afterInitialize.selector;
482 }
```

The hook's entire fee engine then assumes what is true only for pools the protocol created: that `currency0` is native ETH. `_beforeSwap` checks this explicitly (line 760, `Currency.unwrap(key.currency0) == address(0)`) — but `_afterSwap` does not. It decides the sell leg applies purely from swap direction and exactness:

```
635 bool exactInput = params.amountSpecified < 0;
636 bool unspecifiedIsEth = exactInput ? (!params.zeroForOne) : (params.zeroForOne);
637 if (!unspecifiedIsEth) return (BaseHook.afterSwap.selector, 0);
638 ...
639 uint256 fee = _takeEthFee(key, sender, ethAmount, false);
```

and `_takeEthFee` takes `key.currency0` — *whatever that token is* — then credits the proceeds to ETH-denominated counters:

```
961 poolManager.take(key.currency0, address(this), total); // an arbitrary ERC20
962 ...
963 else _routeEthFee(baseFee); // -> relaunchETH += ...
```

Exploit path. One transaction, no capital at risk:

1. Deploy two worthless ERC-20s, `A` and `B`.
2. `poolManager.initialize` the pool `(A, B)` naming the Cauldron hook. Both are ERC-20s, so *neither* leg is ETH.
3. Add your own liquidity, then swap `B→A` exact-input. Now `zeroForOne == false` and `exactInput == true`, so `unspecifiedIsEth` is `true` and the hook charges its "ETH" fee — in units of `A`.
4. `relaunchETH` is now a large number backed by nothing.

Measured on a Sepolia fork (`test/audit/AuditPoC.t.sol::test_PoC_PhantomRelaunchEth_BricksRelaunch`):

`relaunchETH` after one attacker swap	989,374,614,163,774,780,306 (~0.99 "ETH")
Hook's actual ETH balance	0
`releaseRelaunchETH()`	**reverts** `"ETH transfer failed"`

Impact. `CauldronRegistry.relaunch()` calls `releaseRelaunchETH()` whenever `relaunchETH > 0` (`CauldronRegistry.sol:615`), and that call is *not* wrapped in try/catch — `require(ok, "ETH transfer failed")` propagates. So the protocol's central, permissionless, self-funding function reverts forever. There is no reset path: `relaunchETH` is only ever zeroed inside the function that now always reverts. Because `emergencySweep` moves ETH but not the counter, even the break-glass admin cannot clear it. The eternal machine stops being eternal, and every generation's holders lose the migration path that depends on relaunch. The attack costs one pool creation plus gas and can be repeated after any hypothetical manual reset.

Fix. Restrict adoption to pools the protocol created, and defend in depth on the fee path:

```
// 1. Only the registry's own pools are ever tracked.
function _afterInitialize(address sender, PoolKey calldata key, uint160, int24)
    internal override returns (bytes4)
{
    if (sender != registry) return BaseHook.afterInitialize.selector; // ignore
    if (Currency.unwrap(key.currency0) != address(0)) revert ZeroAddress();
    PoolId id = key.toId();
    trackedPools[id] = true;
    _lastUpdateBlock[id] = block.number;
    poolInitBlock[id] = block.number;
    emit PoolTracked(id);
    return BaseHook.afterInitialize.selector;
}

// 2. Belt and braces: never charge the ETH fee on a non-ETH pool.
// In _afterSwap, before the fee block:
if (Currency.unwrap(key.currency0) != address(0) || !trackedPools[id]) {
    return (BaseHook.afterSwap.selector, 0);
}
```

Note that returning the selector without tracking (rather than reverting) is deliberate: reverting in `afterInitialize` would let anyone grief *pool creation*, whereas ignoring the pool simply declines to serve it. Because `registry` is set once and the registry initializes every legitimate pool through `PoolOps`, the `sender` check is exact.

C-01b The legacy buyback spends the hook's ETH inside an attacker-chosen pool — Critical | Confirmed

Location. `CauldronHook.sol:662 _maybeLegacyBuyback` • `:694 legacyBuyStep`

Mechanism. The same missing boundary, but this one moves real money. When the buyback buffer is full, `_afterSwap` fires a self-call at the *top* of the function — before any pool validation — passing the `PoolKey` of *the swap currently executing*:

```
662 function _maybeLegacyBuyback(PoolKey calldata key) private {
663     if (legacyRegistry == address(0) || legacyBuffer < legacyThreshold) return;
664     uint256 gl = gasleft();
665     if (gl > LEGACY_GAS_MIN) {
666         address(this).call{gas: gl - LEGACY_GAS_RESERVE}(
667             abi.encodeWithSelector(this.legacyBuyStep.selector, key) // attacker's key
668         );
669     }
670 }
```

`legacyBuyStep` then spends the *entire* buffer buying `key.currency1` — whatever the attacker chose — and pays for it with the hook's real ETH:

```
700 _inSelfBuy = true;
701 BalanceDelta d = poolManager.swap(key, SwapParams({zeroForOne: true,
702     amountSpecified: -int256(amt), sqrtPriceLimitX96: MIN_SQRT_LIMIT}), "");
703 _inSelfBuy = false;
704 poolManager.settle{value: amt}(); // <-- the hook's ETH leaves
```

Exploit path.

1. Deploy a worthless ERC-20 `E` and initialize `(ETH, E)` naming the hook, priced so that ETH buys an enormous quantity of `E`.
2. Seed it with your own liquidity.
3. Swap a trivial amount (0.001 ETH). In `afterSwap`, the hook sees a full buffer and buys `E` with all of it — inside *your* pool.
4. Withdraw your liquidity. The hook's ETH is now yours; the hook holds worthless `E`.

Measured on a Sepolia fork (`test/audit/AuditPoC.t.sol::test_PoC_LegacyBufferTheft`):

	before	after
Hook ETH balance	5.000 ETH	**0.000 ETH**
Hook `legacyBuffer`	5.000 ETH	0
Attacker ETH	495.000 ETH	**500.000 ETH**
Hook `EVIL` received	—	13.26 (worthless)

Impact. Direct, repeatable theft of every ETH the hook accumulates in its buyback buffer, with a probe swap of arbitrarily small size. Under the shipped configuration (`DeployLaunchpad.s.sol`: `LEGACY_BPS=4000`, `LEGACY_THRESHOLD=0.02 ether`) the buffer is continuously refilled from 40 % of the post-guild fee, so this is a standing tap on protocol revenue that would otherwise back the live collection's NFT floor. The attacker

also poisons `legacyOwedToReserve` with a claim on tokens the hook does not hold in the live token, though `sweepLegacyReserve` caps at the real balance so that part is contained.

Fix. Never spend on a key the caller supplied. Drive the buyback from the protocol's own live pool key:

```
// Store the live key when the registry wires the generation.
PoolKey private _liveKey;
function setLiveKey(PoolKey calldata k) external {
    if (msg.sender != registry) revert OnlyRegistry();
    _liveKey = k;
}

function _maybeLegacyBuyback(PoolKey calldata key) private {
    if (legacyRegistry == address(0) || legacyBuffer < legacyThreshold) return;
    // ONLY ever buy in the protocol's own pool.
    if (PoolId.unwrap(key.toId()) != PoolId.unwrap(_liveKey.toId())) return;
    ...
        abi.encodeWithSelector(this.legacyBuyStep.selector, _liveKey)
}
```

The `trackedPools` fix in C-01a is necessary but *not* sufficient here: an attacker pool would still be tracked if the allow-list were keyed on tracking alone. Pin the key.

C-02 A winning proposal with an out-of-range `nftSupply` freezes the protocol permanently — Critical | Confirmed

Location. `CauldronGovernor.sol:123 propose` • `CauldronRegistry.sol:641,745` • `CauldronCollection.sol:126`

Mechanism. `propose()` validates the name, symbol and metadata source, but places *no bound* on `nftSupply`:

```
137 if (bytes(name).length == 0 || bytes(symbol).length == 0) revert EmptyField();
138 if (mode == MetadataMode.BaseURI) {
139     if (bytes(baseURI).length == 0) revert EmptyField();
140 } else {
141     if (renderer == address(0) || renderer.code.length == 0) revert BadRenderer();
142 }
143 // nftSupply and volumePerNFT: unvalidated
```

`relaunch()` adopts it verbatim (`if (spec.nftSupply > 0) nftSupply = spec.nftSupply;`, line 641) and passes it to the factory, whose collection constructor enforces a hard bound so art ids can never collide with Liquidatoor badge ids:

```
126 if (minter_ == address(0) || maxSupply_ == 0 || maxSupply_ >= LIQUIDATOR_ID_BASE)
127     revert BadConfig(); // LIQUIDATOR_ID_BASE == 1_000_000
```

The bound is correct. The problem is *where* it fires. The revert propagates out of `relaunch()`, and because `governor.markConsumed(winId)` executed earlier *in the same transaction* (line 643), it is rolled back too. The poisoned proposal is never retired, so it wins again on the next attempt, forever.

Exploit path. Any MiFren holder proposes a brew with `nftSupply = 1_000_000` and accumulates enough votes to lead. It need not even be malicious — a proposer who wants a one-million-piece collection triggers it by accident. When the generation dies, every `relaunch()` call reverts.
Verified on a Sepolia fork (`test/audit/AuditPoC4.t.sol`): `relaunch()` reverts `CauldronCollection.BadConfig`, `markConsumed` is rolled back (`gov.consumed() == false`), the generation stays at 1, and a second attempt reverts identically.

Impact. Permanent, unrecoverable freeze of the eternal machine. No relaunch means: no new generation, no migration target for existing holders, no perp `syncGeneration`, and the dead pool's LP stays locked in a zero-volume pool. *The only escape is `setGovernor`, which is `onlyOwner`* — and `DeployLaunchpad.s.sol:248` transfers registry ownership to the `MiFrensGenesis` presale contract, which exposes no call that would forward to it. So on the shipped deployment there is no recovery short of `migrateToSuccessor` — a full V2 migration behind the emergency timelock. My PoC demonstrates recovery via `setGovernor` only to prove that is the sole exit.

Fix. Validate at the boundary, and make relaunch resilient to a bad winner:

```
// 1. CauldronGovernor.propose -- reject what can never be launched.
uint256 constant MAX_NFT_SUPPLY = 100_000; // << LIQUIDATOR_ID_BASE
if (nftSupply > MAX_NFT_SUPPLY) revert EmptyField();

// 2. CauldronRegistry.relaunch -- clamp defensively as well, so a governor
// swapped in later cannot re-introduce the freeze.
if (spec.nftSupply > 0) {
    nftSupply = spec.nftSupply > MAX_NFT_SUPPLY ? MAX_NFT_SUPPLY : spec.nftSupply;
}
```

The same reasoning applies to every other field of `BrewSpec` that reaches a constructor. `renderer` is already checked for code size at proposal time, but note it is checked *again* in the collection constructor — a renderer that self-destructs between proposal and relaunch would reproduce this exact freeze. Clamping in `relaunch` covers both.

H-01 The collection's `deployer` is the factory, so the entire legacy-floor recycle path is unreachable — High | Confirmed

Location. `CauldronFactory.sol:37` • `CauldronCollection.sol:134,296` • `PoolOps.sol:716,740`

Mechanism. `CauldronCollection` records `deployer = msg.sender` in its constructor (line 134). Because collections are created *by the factory* (`new CauldronCollection(...)` at `CauldronFactory.sol:37`), `deployer` is the **factory address**, not the registry. But the custody entrypoint the legacy-floor design depends on is gated on `deployer`:

```
296 function custodyTransfer(address from, address to, uint256 tokenId) external {
297     if (msg.sender != deployer) revert OnlyVault(); // deployer == the FACTORY
298     _transfer(from, to, tokenId);
299 }
```

while the caller is always the registry:

```
712 if (ICollectionOps(collection).ownerOf(tokenId) != caller) revert("not owner");
713 uint256 payout = ILedgerOps(ledger).redeem(gen, mintedNow);
714 ICollectionOps(collection).custodyTransfer(caller, address(this), tokenId); // reverts
```

The factory exposes only `deployBrew` and `deployVault` — there is no forwarding function — so *no address in the system* can pass the gate.

Impact. Three separate consequences, all confirmed by test (`test/audit/AuditPoC.t.sol::PoC_CollectionCustodyLockout`):

1. `recycleCollectionNFT` and `buyCollectionNFT` always revert. The collection legacy floor — an entire subsystem with its own ledger, crystallisation logic and 2× buyback ratchet — is dead code in production. NFT holders can never redeem the floor their collection's own volume paid for. Notably `ledger.redeem` is called *before* the reverting transfer, so the whole transaction rolls back and the ledger is not corrupted — the failure is clean, just total.
2. Every other `deployer`-gated admin function is permanently unreachable: `setMetadata` (so a broken renderer can never be repointed, defeating its stated purpose), `setTransferValidator` (so ERC-721C royalty enforcement can never be switched on), `setLiquidatorURI`, and `setRarityOdds`.
3. The genesis path is *not* affected — `MiFrensGenesis.custodyTransfer` is correctly gated on `registry` (line 337), which is why OG redemption works and the discrepancy is easy to miss.

Fix. Pass the registry explicitly rather than inferring it from `msg.sender`. The `Config` struct already carries it:

```
// CauldronCollection: take the controller as a constructor argument.
constructor(..., address registry_, ...) ERC721(name_, symbol_) {
    ...
    deployer = registry_; // was: msg.sender (the factory)
}

// CauldronFactory.deployBrew: the vault wiring must then happen through a
// path the factory still controls, so do it before handing over -- or keep a
// separate `configurator` slot the factory holds and `deployer` for the registry:
// address public immutable configurator; // = msg.sender (factory), setVault/setRoyalty
// address public immutable deployer; // = c.registry, custodyTransfer/setMetadata
```

The two-slot variant is the safer refactor: the factory keeps exactly the two rights it needs at deploy time (`setVault`, `setRoyalty`) and the registry gets custody and the governed art/validator setters. Add a regression test asserting `col.deployer() == address(registry)`.

H-02 `CauldronSeeder` never resets `complete`, so generation 2 onward never streams — High | Confirmed

Location. `CauldronSeeder.sol:125 startSeed` • `:182 _pendingStep` • `:360 withdrawAll`

Mechanism. The seeder is explicitly designed to be persistent — one instance serves every generation ("*Deployed once (pointed at this registry) and reused every generation*", `CauldronRegistry.sol:168`). Teardown sets three flags:

```
function withdrawAll(address to) external onlyRegistry lock returns (...) {
    poolManager.unlock(abi.encode(ACT_WITHDRAW, to));
    ...
    delete ranges;
    seeding = false;
    complete = true; // <-- sticky
}
```

but `startSeed` resets only `seeding`:

```
    seeding = true;
    // `complete` : NOT reset
    // `_basePlaced` : NOT reset
```

and every placement is gated on `complete`:

```
function _pendingStep() private view returns (uint256) {
    if (!seeding || complete) return 0; // <-- always 0 from gen 2 onward
    ...
}
```

Exploit path. No attacker required — this is the *normal* second launch. Verified on a Sepolia fork (`test/audit/AuditPoC.t.sol::test_PoC_SecondCampaignNeverStreams`):

Gen-1 campaign streams to 100 %	`deployedWad == 1e18`
After teardown, gen-2 `startSeed`	`isComplete() == true` **immediately**
After the full window elapses + `poke()`	`deployedWad == 0.1e18` (unchanged)
Gen-2 token stranded in the seeder	91,500,000 of 100,000,000 (**91.5 %**)
Gen-2 ETH stranded in the seeder	9.15 of 10 ETH (**91.5 %**)

Impact. From generation 2 onward, only the 10 % seed floor ever reaches the pool. The remaining ≈90 % of the active tranche — both ETH and tokens — sits in the seeder contract for the entire life of the generation. Consequences compound:

- **Catastrophically thin liquidity.** The pool trades on a tenth of its intended depth for its whole life, so ordinary trades suffer roughly an order of magnitude more slippage.
- **Death threshold becomes trivial to hit.** Thin liquidity suppresses volume, and volume is what keeps the generation alive.
- **Perps are silently disabled.** `_basePlaced` is also sticky, so `_placeBase` never runs and the pool has no spot-straddling base — `activeEthDepth()` collapses and `maxLeverage()` bottoms out.
- Funds are *not* lost — the next `withdrawAll` recovers them — but they are unavailable for the entire generation.

The repository's own `LifecycleE2E` test asserts `seeder.seeding()` after the gen-2 relaunch but never re-checks `deployedWad`, which is why this survived to now.

Fix. One line, plus a hygiene reset:

```
function startSeed(SeederConfig calldata cfg) external payable onlyRegistry lock {
    if (seeding) revert AlreadySeeding();
    ...
    seeding = true;
    complete = false; // <-- FIX: a fresh campaign is not complete
    _basePlaced = false; // <-- FIX: lay a new two-sided base for the new pool
    _lastEthOut = 0; // hygiene: stale return-value scratch
    _lastTokenOut = 0;
    ...
}
```

Then extend `ProgressiveSeed.t.sol` to run *two* progressive generations back to back and assert `deployedWad() == 1e18` on the second.

H-03 `claimByBurn` burns the full amount but silently accepts a short delivery

High | Confirmed

Location. `CauldronRegistry.sol:835 claimByBurn` • `PoolOps.sol:524 claimFromReserve` • `:601 migrateOne`

Mechanism. `claimFromReserve` caps the withdrawal at the position's available liquidity and returns whatever it managed to take:

```
uint128 liquidity = ReserveLib.liquidityForTokenOut(tickLower, tickUpper, amount);
if (liquidity == 0) return 0;
uint128 have = pm.getPositionLiquidity(positionId);
if (liquidity > have) liquidity = have; // <-- silently truncate
if (liquidity == 0) return 0;
...
taken = IERC20(...).balanceOf(recipient) - tokBefore; // may be << amount
```

`migrateOne` burns *first* and never compares:

```
601 function migrateOne(..., uint256 amount, ReserveRef memory r) public returns (uint256 got) {
602     ICauldronBurn(prevToken).burn(from, amount); // full amount destroyed
603     got = claimFromReserve(pm, r.positionId, ..., amount, from); // may deliver less
604 }
```

The registry's own comment asserts the opposite — "*A short reserve reverts, rolling the burn back with it*" (line 851) — which is exactly the behaviour the code does *not* implement.

Exploit path. Verified against live v4 (`test/audit/AuditPoC2.t.sol::PoC_ReserveUnderDelivery`): a reserve holding 100 tokens, asked for 10,000, delivers 99.99 and returns normally — a 99 % shortfall with no revert.

Requested	$10{,}000 \times 10^{18}$
Delivered	$99.999\ldots \times 10^{18}$
Shortfall	$9{,}900 \times 10^{18}$ **destroyed**

In the benign case the gap is quantization dust: my lifecycle test measured **276 wei** lost on a 2.8×10^{26} claim (`test_KnownDeviation_MigrationIsNotExactlyOneToOne`). That is economically irrelevant *but it proves the check is absent*, so the unbounded case is reachable whenever the reserve is short.

When is the reserve short? Relaunch sizes it as `TOTAL_SUPPLY - newActive`, intended to cover migration plus genesis plus legacy. Several paths can under-size it: the degenerate guard at line 676 (`if (newActive == 0) newActive = GEN1_ACTIVE_TOKENS`) discards the sizing entirely; `PoolOps.crystallizeCollection` can add entitlement after sizing; and cumulative dust across many generations compounds in the same direction.

Impact. A migrating holder can have their old tokens burned and receive materially less than 1:1, with no revert and no recourse. The loss is permanent (the old tokens are destroyed). This directly violates the advertised "migration is conserved 1:1" property, and because `MigrationVesting._pullAndVest` wraps the same call, escrow users are exposed identically — though note the escrow *does* verify its own balance delta (line 186), which is the right pattern applied one layer too high.

Fix. Make short delivery a revert, not a silent truncation:

```
// PoolOps.migrateOne -- the migration path must be exact.
function migrateOne(IPositionManagerOps pm, address prevToken, address from,
                    uint256 amount, ReserveRef memory r) public returns (uint256 got) {
    ICauldronBurn(prevToken).burn(from, amount);
    got = claimFromReserve(pm, r.positionId, r.key, r.tickLower, r.tickUpper, amount, from);
    // Liquidity math rounds DOWN, so allow a wei-scale tolerance but nothing more.
    require(got + 1e12 >= amount, "reserve short"); // reverts -> burn rolls back
}
```

Apply the same guard in `PoolOps.recycleCollection` and in `RedemptionExt.redeemOgFren`, where the NFT has already moved to the treasury before the reserve is drawn. For the redemption paths a strict `require(taken >= payout)` is appropriate, since those amounts are computed from the ledger rather than supplied by the caller.

H-04 A trader contract that rejects ETH can freeze the perp engine on a dead generation — High | Confirmed

Location. `PerpEngine.sol:993 _sendEth • :858 _settle • :700 forceCloseAllDead • :725 syncGeneration`

Mechanism. Settlement pays the trader with a send that reverts on failure:

```
993 function _sendEth(address to, uint256 amount) internal {
994     (bool ok, ) = to.call{value: amount}(""); if (!ok) revert EthSend();
995 }
```

```
858 if (residual > 0) _sendEth(p.trader, residual); // trader controls this
```

A contract whose `receive()` reverts therefore makes its own position permanently unsettleable — by `close`, by `liquidate`, by `forceCloseDead`, and critically by the batch:

```
709 while (openCount != 0 && iters < 96) {
710     uint256 id = _openIds[0];
711     _settle(id, positions[id], 0, MODE_DEATH, msg.sender); // reverts -> whole batch reverts
712 }
```

and `syncGeneration` refuses to re-arm while anything is open:

```
728 if (openCount != 0) revert PositionsOpen();
```

Exploit path. Verified on a Sepolia fork (`test/audit/AuditPoC3.t.sol::test_PoC_RejectingTraderFreezesTheEngine`):

1. Deploy a contract with a reverting `receive()`; open a 0.05 ETH long through it. (Cost: the 6.9 % open fee.)

2. The generation dies. An honest position force-closes fine.
3. `forceCloseDead(griefId)` reverts `EthSend`.
4. `forceCloseAllDead()` reverts `EthSend` — the entire batch.
5. `relaunch()` still succeeds, because the registry try/catches the force-close (`CauldronRegistry.sol:737`) — good defensive design — but the position survives the rebirth.
6. `syncGeneration()` reverts `PositionsOpen`, permanently.

Observed end state: `syncedToken` = the *dead* generation-1 token while `registry.currentToken()` is generation 2, with 50,000,000 tokens of short inventory denominated in a token nobody trades.

Impact. For roughly the cost of one open fee, an attacker permanently disables the perp engine across every future generation:

- `plvToken` stays denominated in a dead token; the entire short-side inventory is stranded and `PerpVault` token stakers cannot be made whole (`withdrawPlvTokenTo` pays out the dead token).
- `openShort` still reads `plvToken` as available, so shorts could be opened against inventory the engine cannot actually deliver on the live pool.
- The advertised "stake & chill" auto-migration silently stops working for everyone.

Fix. Never let an untrusted recipient's revert block settlement. Use a pull pattern for the trader payout, exactly as the hook already does for `proposerOwed`:

```
mapping(address => uint256) public payoutOwed;

function _sendEthSafe(address to, uint256 amount) internal {
    if (amount == 0) return;
    (bool ok, ) = to.call{value: amount, gas: 30_000}("");
    if (!ok) payoutOwed[to] += amount; // credit instead of reverting
}

function claimPayout() external nonReentrant returns (uint256 amt) {
    amt = payoutOwed[msg.sender];
    if (amt == 0) revert ZeroValue();
    payoutOwed[msg.sender] = 0;
    (bool ok, ) = msg.sender.call{value: amt}("");
    if (!ok) revert EthSend();
}
```

Use `_sendEthSafe` for the trader residual (line 858) and the keeper reward (line 846/856). Keep the reverting `_sendEth` only for the protocol's own trusted sinks (`dividend`, `treasury`) where a failure genuinely indicates misconfiguration — though crediting there too would be more robust still.

H-05 The first token-side staker captures the entire previously-accrued yield pot

High | **Confirmed**

Location. `PerpVault.sol:177 _syncTokYield`

Mechanism. Short-side fees accrue in the engine as `tokYieldEth` with a monotonic watermark `tokYieldCumulative`. The vault folds new accrual into a per-share accumulator:

```
177 function _syncTokYield() internal {
178     uint256 cum = engine.tokYieldCumulative();
179     if (cum == lastTokYieldCum || tokShares == 0) return; // <-- watermark NOT advanced
180     accEthPerTokShare += FullMath.mulDiv(cum - lastTokYieldCum, ACC, tokShares);
181     lastTokYieldCum = cum; // "the first staker collects"
182 }
```

When `tokShares == 0` the function returns *without* advancing `lastTokYieldCum`. So all yield accrued during the zero-stake period stays in the delta and is divided over whatever share count exists at the next sync. The trailing comment ("*the first staker collects*") shows the behaviour was noticed; the finding is that its magnitude is unbounded.

Exploit path. A watcher monitors `tokYieldEth`. When it is large and `tokShares == 0` (true at launch, and again after any full unstake), they deposit the minimum that mints one share and immediately claim. Verified in `test/audit/AuditPoC.t.sol::PoC_PerpVaultYieldCapture`:

Pot accrued while `tokShares == 0`	10.0 ETH
Sniper deposit	1,000 tokens
Sniper claim	**10.0 ETH** (100 % of the pot)
Whale deposit, 1000× larger, moments later	1,000,000 tokens
Whale pending yield	**0**

Impact. The entire short-side yield pot is transferred to whoever deposits first after a zero-stake window, in proportion to nothing. The window is guaranteed to occur at least once (launch) and recurs after any complete unstake. Deposit size is irrelevant — only ordering matters — so the capture is essentially free. There is no cap: the longer the protocol runs before the token side is seeded, the larger the prize. This is a two-sided problem: it also means honest first stakers receive a windfall they did not earn, distorting the vault's advertised yield.

Fix. Advance the watermark even when there are no shares, and let the unattributable yield fall to a defined sink:

```
function _syncTokYield() internal {
    uint256 cum = engine.tokYieldCumulative();
    if (cum == lastTokYieldCum) return;
    if (tokShares == 0) {
        // Nobody is staked: this yield has no claimant. Advance the watermark so
        // it can never be back-paid to a future first-mover.
        lastTokYieldCum = cum;
        emit UnattributedYield(cum - lastTokYieldCum);
        return;
    }
    accEthPerTokShare += FullMath.mulDiv(cum - lastTokYieldCum, ACC, tokShares);
    lastTokYieldCum = cum;
}
```

The ETH itself remains in the engine's `tokYieldEth` pot; add an owner/timelock path to route the orphaned balance to the treasury or to donate it into `insuranceEth`. Note that `pendingTokYield` (line 286) mirrors the same logic and must be updated in lockstep or the UI will disagree with the contract.

M-01 `setRegistryOverride` defeats the one-shot registry lock the code documents — Medium | Confirmed

Location. `CauldronHook.sol:1188 setRegistry • :1200 setRegistryOverride`

Mechanism. `setRegistry` is one-shot and its comment is explicit about why: "*a mutable setter would let the hook owner repoint `registry` to an address they control and drain `relaunchETH` via `releaseRelaunchETH()` (audit F1). Set once at deploy, then locked forever.*" Twelve lines later:

```
1200 function setRegistryOverride(address _registry) external onlyOwner {
1201     if (_registry == address(0)) revert ZeroAddress();
1202     registry = _registry; // no one-shot guard
1203 }
```

Verified in `test/audit/AuditPoC2.t.sol::PoC_RegistryOverrideDrain`: `setRegistry` reverts `RegistryAlreadySet` while `setRegistryOverride` succeeds, after which the original registry gets `OnlyRegistry` on `releaseRelaunchETH`.

Impact. The hook owner can re-point `registry` to any address and immediately call `releaseRelaunchETH()` to take the accumulated relaunch reserve. It also grants `setCollection`, `setVault`, `setNftCurveFrom`, `forceClosePerps` and `sweepLegacyReserve`. Mitigating context: `DeployPerp.s.sol:106` transfers hook ownership to a `TimelockController`, so on the shipped mainnet configuration this is a delayed, publicly-visible governance action rather than an instant rug. That is what keeps it Medium rather than High. The function is legitimately needed for the V2 handoff — the issue is that it is unconstrained and that the adjacent comment claims a protection the contract does not provide.

Fix. Route the override through the same arm/delay/veto machinery that protects the registry's own custody actions, and correct the misleading comment:

```
address public pendingRegistry;
uint256 public registrySwapReadyAt;
uint256 public constant REGISTRY_SWAP_DELAY = 7 days;

function proposeRegistryOverride(address _r) external onlyOwner {
    if (_r == address(0)) revert ZeroAddress();
    pendingRegistry = _r;
    registrySwapReadyAt = block.timestamp + REGISTRY_SWAP_DELAY;
    emit RegistryOverrideProposed(_r, registrySwapReadyAt);
}

function executeRegistryOverride() external onlyOwner {
    if (registrySwapReadyAt == 0 || block.timestamp < registrySwapReadyAt) revert Timelocked();
    // Drain the reserve to the OUTGOING registry first, so a swap can never
    // capture ETH the previous registry accrued.
    if (relaunchETH > 0) releaseRelaunchETH();
    registry = pendingRegistry;
    pendingRegistry = address(0); registrySwapReadyAt = 0;
}
```

M-02 `winner()` returns the live vote leader, so relaunch is front-runnable
Medium | **Confirmed**

Location. `CauldronGovernor.sol:211 winner` • `:181 vote` • `CauldronRegistry.sol:635`

Mechanism. There is no voting period, no deadline and no quorum. `winner()` returns whichever unconsumed proposal currently has the most votes, and `relaunch()` reads it at execution time. Since `relaunch()` is permissionless and therefore public in the mempool, a voter can place their vote in the same block, ahead of it.
Verified in `test/audit/AuditPoC2.t.sol::PoC_GovernorLiveWinner`: the honest proposal leads; one whale vote flips `winner()` to the attacker's proposal, along with `spec.proposer`.

Impact. Whoever holds the largest checkpointed MiFrens balance decides every relaunch unilaterally, at the last possible instant, with no opportunity for anyone to respond. The winning `proposer` also captures the hook's proposer fee slice for that entire generation (`CauldronRegistry.sol:658`). Combined with **C-02**, front-running also means a malicious winner can be inserted at the instant of relaunch. The snapshot mechanism at line 191 correctly prevents the classic transfer-and-revote attack — the gap is purely the absence of a deadline.

Fix. Give proposals a voting window and freeze the winner before it can be consumed:

```
uint256 public constant VOTING_PERIOD = 3 days;
// Proposal gains: uint256 votingEndsAt; (set to block.timestamp + VOTING_PERIOD)

function vote(uint256 proposalId) external {
    ...
    if (block.timestamp > p.votingEndsAt) revert VotingClosed();
    ...
}

function _bestUnconsumed() private view returns (uint256) {
    // Only proposals whose voting window has CLOSED are eligible to win.
    ... if (p.votingEndsAt >= block.timestamp) continue; ...
}
```

This also gives holders a visible window in which to observe and respond to a proposal before it can be launched, which is the point of having governance at all.

M-03 The rarity reveal offers two independent draws and the holder picks the better
Medium | **Confirmed**

Location. `CauldronCollection.sol:196 reveal` • `MiFrensGenesis.sol:409 reveal`

Mechanism. `reveal()` rolls rarity from the mint block's hash, with a deterministic fallback once that hash ages out of the 256-block window:

```
bytes32 bh = blockhash(mb);
if (bh == 0) bh = keccak256(abi.encodePacked("CAULDRON_REVEAL_FALLBACK", mb, tokenId));
uint8 rarity = _rollRarity(uint256(keccak256(abi.encodePacked(bh, tokenId, address(this)))));
```

Reveal timing is entirely the holder's choice, so they get *two* independent outcomes: the blockhash roll (readable from block `mb+1`) and the fallback roll (computable at mint time, since it depends only on `mb` and `tokenId`). They simply wait if the early roll is worse. Verified in `test/audit/AuditPoC.t.sol::PoC_RarityDoubleRoll`.

Impact. With the default tiers (Common 79 %, Rare 15 %, Epic 5 %, Ultra 1 %), taking the maximum of two draws lifts P(at least Rare) from 21 % to $1 - 0.79^2 \approx 37.6\,\%$ and P(Ultra) from 1 % to $\approx 1.99\,\%$ — roughly doubling the top tier. Every patient holder gets this; only naive ones do not. The fallback is also *fully predictable at mint time*, so a minter knows before revealing whether waiting 256 blocks guarantees them a specific rarity. The comment acknowledges a weakened fallback but frames the holder's advantage as "a one-alternative choice" — which is precisely a free re-roll.

Fix. Make the reveal seed independent of when the holder reveals:

```
function reveal(uint256 tokenId) external {
    if (ownerOf(tokenId) != msg.sender) revert OnlyMinter();
    if (revealed[tokenId]) return;
    uint256 mb = mintBlockOf[tokenId];
    if (block.number <= mb) revert NotReady();
    bytes32 bh = blockhash(mb);
    // Expired seed: DO NOT substitute a second, holder-predictable roll.
    // Anyone may re-anchor the token to a fresh future block instead, so the
    // holder still cannot choose between two known outcomes.
    if (bh == 0) { mintBlockOf[tokenId] = uint48(block.number); revert NotReady(); }
    ...
}
```

Re-anchoring keeps every token revealable indefinitely while ensuring exactly one unknowable draw. If a permissionless `revealMany(uint256[])` keeper is added, most tokens will be revealed inside the window anyway.

M-04 Crystal-ticket resolution shares the same predictable 256-block fallback

Medium | **Confirmed**

Location. `CauldronHook.sol:1530 _resolveTickets`

Mechanism. Identical pattern in the gacha lottery:

```
1539 bytes32 bh = blockhash(b.commitBlock);
1540 if (bh == 0) bh = keccak256(abi.encodePacked("CAULDRON_TICKET_FALLBACK", b.commitBlock, bi));
1541 ...
1542 uint256 roll = uint256(keccak256(abi.encodePacked(bh, player, bi, r))) % 10_000;
```

Here the exposure is different in kind. Resolution is permissionless and FIFO, so a player cannot choose *their* outcome directly — but the fallback seed is computable from `commitBlock` and the batch index alone, both known at commit time. A player who lets their batch age past 256 blocks knows their outcome in advance and can decide whether to call `resolveTickets` at all, or to wait for a state in which a loss costs less (e.g. after mint-out, when losses are free). Because `missStreak` feeds a pity counter, selectively resolving batches is a real lever on future odds.

Impact. Weakened grind-resistance for the gacha and a manipulable pity counter. The severity is limited by the FIFO cursor — a player cannot resolve past someone else's unresolved batch — and by `outstandingOf` capping total wins at remaining supply, so the collection cap is never breached. It also affects the honest path: the hook calls `resolveTickets(300)` during `relaunch` (`CauldronRegistry.sol:603`), which is exactly when old batches are most likely to have aged out.

Fix. Do not substitute a computable seed. Skip aged batches and let them be re-anchored:

```
bytes32 bh = blockhash(b.commitBlock);
if (bh == 0) {
    // Seed expired: re-anchor to a fresh future block rather than rolling from a
    // value the player already knows. Costs one SSTORE, preserves fairness.
    b.commitBlock = uint48(block.number);
    break; // FIFO: stop here, resume next call
}
```

Combined with the existing in-swap resolution (`NATIVE_RESOLVE_MAX`) and the router's `resolveTickets(30)`, batches are resolved within a handful of blocks in practice, so re-anchoring should be rare.

M-05 `skimInsurance` protects only `insuranceFloor`, which defaults to zero

Medium | **Confirmed**

Location. `PerpEngine.sol:1213 skimInsurance • :142 insuranceFloor`

Mechanism.

```
1213 function skimInsurance(uint256 amount, address to) external onlyOwner {
1214     if (to == address(0)) revert BadParam();
1215     if (insuranceEth < insuranceFloor + amount) revert BadParam(); // keep the floor
1216     insuranceEth -= amount;
1217     _sendEth(to, amount);
1218 }
```

The guard is only as strong as `insuranceFloor`, which is declared `uint256 public insuranceFloor;` with no initialiser — so **0** — and is documented as "`0 = off`" (line 141). Neither `DeployPerp.s.sol` nor any test sets it. With a zero floor the condition reduces to `insuranceEth >= amount`, i.e. no protection at all.

Impact. The owner can withdraw the entire bad-debt buffer, including the 10 % of every routed fee (`insuranceBps`) that accumulated specifically to protect depositor principal. This directly undercuts the invariant the buffer exists to support: with insurance drained, `_absorbPlvLoss` socialises the next short shortfall straight onto `plv` — LP principal. It also silently disables the circuit breaker at line 487, since `insuranceFloor == 0` makes that check a no-op. Contrast with `setVault`, whose drain guard is genuinely strong; the two protections are inconsistent in strength.

Fix. Derive the protected minimum from live risk rather than a zero-defaulted knob:

```
function skimInsurance(uint256 amount, address to) external onlyOwner {
    if (to == address(0)) revert BadParam();
    // Protect the greater of the configured floor and a risk-based minimum
    // scaled to live open interest, so the buffer can never be emptied while
    // positions depend on it.
    uint256 riskMin = ((longOiEth + _quoteEth(shortOiToken)) * maintenanceBps) / BPS;
    uint256 protect = insuranceFloor > riskMin ? insuranceFloor : riskMin;
    if (insuranceEth < protect + amount) revert BadParam();
    insuranceEth -= amount;
    _sendEth(to, amount);
}
```

Separately, set a non-zero `insuranceFloor` in `DeployPerp.s.sol` so the opens circuit-breaker is actually armed.

M-06 Forged frens never break their dividend enchantment on transfer Medium | Confirmed

Location. `MiFrensGenesis.sol:532 _update` • `MiFrensDividend.sol:150,179,224`

Mechanism. The dividend's eligibility cap is `MAX_TOKEN` (the full art supply, e.g. 2222), so *forged* frens above `GENESIS_SUPPLY` can enchant and earn:

```
179 if (tokenId == 0 || tokenId > MAX_TOKEN) revert NotShare(); // forged frens allowed
```

But the collection only notifies the dividend for *genesis* ids:

```
532 if (from != address(0) && tokenId <= GENESIS_SUPPLY && dividend != address(0)) {
533     try IMiFrensDividendHook(dividend).onMiFrenTransfer(tokenId, from) {} catch {}
534 }
```

so a forged fren's `enchantedBy` is never cleared and its active share is never freed.

Exploit path. Verified in `test/audit/AuditPoC2.t.sol::PoC_ForgedFrenSpellLeak`:

1. Seller enchants forged fren #3; `activeShares == 2`.
2. Seller sells #3 on the open market to a buyer.
3. `activeShares` is still 2 and `enchantedBy(3)` is still the *seller*.
4. 2 ETH of fees arrive and are split two ways.
5. The buyer calls `claim(3)` → reverts `NotEnchanted`. They own it and cannot claim.
6. The buyer re-casts. `_castSpell`'s stale branch (line 194) credits the accrual *earned while the buyer owned the NFT* to `owed[seller]`.
7. The seller withdraws **1 ETH** of the buyer's dividend. Confirmed by assertion.

Impact. Three distinct harms: (i) a phantom share dilutes every honest holder for as long as nobody re-casts; (ii) the new owner cannot claim until they re-cast, and loses everything accrued before that; (iii) the previous owner is paid ETH earned by someone else's ownership — a direct, repeatable transfer of value backwards. A seller can farm this by repeatedly selling enchanted forged frens. The genesis tranche is unaffected, which is why the flaw is invisible until iteration #2 forges its first fren.

Fix. Notify for every dividend-eligible id, not just genesis:

```
// MiFrensGenesis._update -- the dividend decides eligibility, not the collection.
if (from != address(0) && dividend != address(0)) {
    try IMiFrensDividendHook(dividend).onMiFrenTransfer(tokenId, from) {} catch {}
}
```

`onMiFrenTransfer` is already a safe no-op for an un-enchanted id (line 227), and it is `try/catch`'d, so widening the call is safe. Note that Liquidatoor badge ids ($\geq 10^6$) also route through `_update`; they are never enchanted, so they hit the same early return. Keep `everMoved` gated on `GENESIS_SUPPLY` — that flag is specifically about the OG re-enchant fee.

L-01 `MAX_RANGES` can halt the progressive stream mid-launch Low | Confirmed

Location. `CauldronSeeder.sol:344 _track` • `:95 MAX_RANGES`

Each placement mints into a band derived from the *current* tick, so a trending price produces new `(lo,hi)` pairs. `_track` reverts `BandCap` at 64 distinct ranges, which halts `poke()` and makes the in-swap nudge a silent no-op — streaming stops for the rest of the launch.

I attempted to reach the cap on a fork with an aggressively trending price and 60 pokes and observed only 5–19 distinct ranges, because `SeedLib.askBand` aligns to 2000-tick bands and a monotone walk revisits them. So the cap is genuinely hard to hit — but it is reachable in principle over a long window with a volatile price, and the failure mode (silent halt) is identical to **H-02**. Funds are never lost: `withdrawAll` recovers both the tracked positions and the un-streamed balance.

Fix. Degrade instead of halting: on cap, place into the nearest *already-tracked* band rather than reverting.

```
function _trackOrReuse(int24 lo, int24 hi) private returns (int24, int24) {
    uint256 n = ranges.length;
    for (uint256 i; i < n; i++) if (ranges[i].lo == lo && ranges[i].hi == hi) return (lo, hi);
    if (n >= MAX_RANGES) { Range storage r = ranges[n - 1]; return (r.lo, r.hi); } // reuse
    ranges.push(Range(lo, hi));
    return (lo, hi);
}
```

L-02 The anti-sniper surtax jitter is predictable one block ahead Low | Confirmed

Location. `CauldronHook.sol:912 _defaultSurtaxBps`

```
929 uint256 rnd = uint256(keccak256(abi.encodePacked(
930     blockhash(block.number - 1), PoolId.unwrap(id), block.number))) % (maxBps + 1);
```

The comment's reasoning is sound — `prevrandao` is a constant on Arbitrum/Orbit, so the previous blockhash is the better source. But `blockhash(block.number - 1)` is known to everyone during block N, so a sniper submitting into block N computes the exact surtax before deciding. They can therefore skip expensive blocks and enter only on cheap ones.

Impact is genuinely limited: the deterministic decay term dominates early (`snipeMaxBps = 9600`) and the jitter only ever *raises* the rate via `max(decayed, jitter)`. The practical gain is skipping the tail of the window. **Fix:** accept it (documented), or fold in a value unknown at submission such as the pool's post-swap tick, or shorten `snipeWindowBlocks` so the decay term dominates throughout.

L-03 `tx.origin` credit attribution mis-attributes smart-account swaps Low | Confirmed

Location. `CauldronHook.sol:537`

```
534 bool tagged = hookData.length >= 32;
535 address player = tagged ? abi.decode(hookData, (address))
536                        : (creditUntaggedSwaps ? tx.origin : address(0));
```

For an ERC-4337 smart account, `tx.origin` is the *bundler*, so a bundler accumulates crystal credit for every user operation it relays; the same applies to Safe transactions relayed by a service. As written the design is deliberate and reasonable for EOAs — credit is proportional to volume, which costs real fees — but the attribution is wrong for an increasingly common class of users, and it will get worse over time.

Fix. Prefer the tagged player, fall back to `sender` when it is a known router, and only then to `tx.origin`; consider an explicit `creditRecipientOf` registry so smart accounts can nominate a beneficiary.

L-04 The tiered NFT tax reads the swap sender, not the trader Low | Confirmed

Location. `CauldronHook.sol:948, :1050`

`_takeEthFee` passes `sender` — the address that called `poolManager.swap`, i.e. the router — to `_getHolderTaxRate`. So a trader routing through `CauldronGachaRouter`, a v4 universal router, or an aggregator is taxed at the router's tier (in practice the default 3 %), regardless of what they hold. Only a direct swap from the holder's own EOA gets the discount.

The hook *does* already decode the real player from `hookData` for credit attribution and for `_isExemptPlayer` — so the information is present, it is simply not used for the tier. Since `nftContract` is `address(0)` in the shipped deploy, the tier system is currently inert, which is why this is Low.

Fix. Resolve the taxed identity the same way exemption does: use the tagged player when the sender is a trusted opener, else the sender.

L-05 The funding index is sized at spot, not at the manipulation-resistant mark Low | Likely

Location. `PerpEngine.sol:426 _pokeFunding`

```
426 uint256 shortEth = _quoteEth(shortOiToken); // SPOT, not markSqrtPriceX96
427 int256 imbalance = int256(longOiEth) - int256(shortEth);
```

Liquidation correctly uses the TWAP mark, but the funding imbalance is valued at spot. An actor who can move spot within a block can therefore bias the funding direction. Three things bound the damage substantially: the index accrues $\propto \Delta t$, so a single-block distortion contributes almost nothing; `_fundingDelta` pins notional to entry collateral × leverage rather than to price; and the result is clamped to $\pm$`maxFundingBps` (50 %) of collateral. I have not constructed a profitable exploit and rate the finding *Likely* rather than Confirmed on that basis.

Fix. Use `_quoteMark(shortOiToken)` for the imbalance so that every economically-significant valuation in the engine uses the same manipulation-resistant source.

L-06 A Liquidatoor badge is silently lost when no collection is wired Low | Confirmed

Location. `PerpEngine.sol:1029 _awardBadge`

```
(bool ok, ) = IPerpHook(hookAddr).collection().call{gas: fwd}(
    abi.encodeWithSelector(ILiquidatorMintable.mintLiquidator.selector, to));
if (ok) minted = 1;
```

A low-level `call` to an address with no code returns `true`. If `hook.collection()` is `address(0)` — which it is between a `setCollection(0)` and the next wiring, and during any window where the hook has no live collection — `ok` is `true`, `minted` is set to 1, and the fallback credit in `badgesOwed` is skipped. The badge is neither minted nor owed. The event even reports `badgeId == 1`, so off-chain indexers record a mint that never happened.

Fix. Check for code before treating success as a mint:

```
address col = IPerpHook(hookAddr).collection();
uint256 minted;
if (col.code.length > 0 && gasleft() > 220_000) {
    uint256 fwd; unchecked { fwd = gasleft() - 120_000; }
    (bool ok, ) = col.call{gas: fwd}(abi.encodeWithSelector(
        ILiquidatorMintable.mintLiquidator.selector, to));
    if (ok) minted = 1;
}
if (minted == 0) { unchecked { badgesOwed[to] += 1; } }
```

`claimLiquidatorBadges` has the mirror-image issue: it reads `col` without a code check and would loop over a code-less address, burning the owed count for nothing. Add the same guard there.

L-07 `CauldronVault.redeem` is unreachable under the unified-floor model Low | Confirmed

Location. `CauldronVault.sol:84` • `CauldronRegistry.sol:781, :805`

Both collection-deployment paths wire `hook.setVault(address(0))`, so the ETH floor share is routed into the token buyback buffer and no ETH ever reaches a `CauldronVault`. Its balance is therefore always zero, `floorPerNFT` returns 0, and `redeem` always reverts `NothingToRedeem`.

The vault is retained deliberately — `crystallizeCollection` reads `outstanding()` for the entitlement base — so this is not dead code, but it presents a redemption surface that can never succeed, and it will confuse integrators and holders reading the contract. **Fix:** either remove `redeem`/`floorPerNFT` in favour of a purpose-named `CollectionSupplyCounter`, or have `redeem` revert with an explicit `UnifiedFloorActive()` error pointing at `recycleCollectionNFT`.

L-08 `migrateToSuccessor` leaves the seeder's liquidity behind Low | Confirmed

Location. `CauldronRegistry.sol:320`

The V2 handoff transfers the active and reserve position NFTs and any loose balances, but does not touch the seeder. On a *progressive* generation `generationPositionId[g] == 0` and the active liquidity lives in the seeder's N core positions, owned by the seeder and recoverable only via `withdrawAll`, which is `onlyRegistry` — the *old* registry.

After a handoff the successor controls the reserve but not the active book. The old registry still can call `withdrawAll`, so nothing is permanently lost, but the migration is not atomic and leaves the two ledgers under different controllers.

Fix: tear the seeder down as part of the handoff.

```
address _seeder = seeder;
if (_seeder != address(0) && ISeeder(_seeder).seeding()) {
    (uint256 e, uint256 t) = ISeeder(_seeder).withdrawAll(address(this));
    e; t; // now included in the loose-balance forward below
}
```

Informational

I-01 — `accumulatedTokenFees` is never written.
`CauldronHook.sol:175,1096`. `withdrawTokenFees` reads a mapping that no code path ever increments (fees are ETH-only by design, per the header comment). Every call reverts `NothingToWithdraw`. Remove both, or implement the token-fee path the ABI advertises.

I-02 — `setGuildBps` cannot restore its own default.
`CauldronHook.sol:1291` caps at 500 while `guildBps` initialises to **1500** (line 352). The setter is a one-way ratchet: once called, the guild share can never be returned to the deployed 15 %. Either raise the cap to 1500 or lower the default.

I-03 — `CauldronSeeder.rescue` has no reachable caller.
`CauldronSeeder.sol:376` is `onlyRegistry`, but `CauldronRegistry` exposes no function that calls it. The escape hatch for an aborted campaign cannot be used. Add a registry-side `rescueSeeder(address)` behind `onlyEmergency`.

I-04 — The `claimed` mapping is vestigial.
`CauldronBase.sol:184` and `CauldronRegistry.sol:1225` (`hasClaimed`) are never written under the reserve-LP model. Harmless but misleading; retained because removing it would shift the shared storage layout and require re-verifying the facet. Note this in the layout documentation rather than removing it casually.

I-05 — `liquidityForTokenOut` reverts at pathological ticks.
`ReserveLib.sol:74`. For launch ticks below roughly −480,000 the reserve band's sqrt-price width becomes small enough that `getLiquidityForAmount1(777M)` exceeds `uint128` and reverts `SafeCastOverflow`. A Cauldron pool prices $\approx 7.77 \times 10^{26}$ token-wei against 10^{18}–10^{21} ETH-wei, giving a strongly *positive* launch tick ($\approx$ +204,000 at genesis), so this is unreachable for the shipped economics. I measured the boundary rather than assuming it, and bounded the fuzz domain accordingly with an explanatory comment.

I-06 — `PerpEngine` has 4 bytes of EIP-170 headroom.
`forge build -sizes` reports 24,572 / 24,576 bytes. Any fix to **H-04**, **L-05** or **L-06** will exceed the limit. Plan the extraction now: `_settle`'s fee/penalty routing and the badge logic are the natural candidates for a linked library, following the `PoolOps` pattern already used by the registry.

I-07 — `Batch` fields are `uint16`-narrow.
`CauldronHook.sol:274`. `count` and `resolved` are `uint16` and `n` is cast without a check at line 1503. `_commitCrystals` bounds `n` by `MAX_MINTS_PER_CALL` (30), so overflow is unreachable today — but the cast is unchecked and would silently truncate if that constant were ever raised. Add `require(n <= type(uint16).max)` or widen the fields. Marked *Suspected* because it depends on a constant rather than on reachable state.

7 Positive observations

It is worth recording what the code gets right, both because it is unusual and because a reader should know which defences they can rely on.

- **The facet split is done correctly**, including the immutable trap, the shared reentrancy-guard slot, and the empty-account delegatecall hazard (§4). All three are easy to get wrong and all three are handled.
- **Fee routing is push-free where the recipient is untrusted.** `activeProposer` is attacker-controlled (anyone can propose), and the code accrues to `proposerOwed` rather than pushing — with a comment that states exactly this reasoning (`CauldronHook.sol:818`). By contrast the perp engine pushes to the trader, which is **H-04**; the hook's pattern is the one to copy.
- **Best-effort calls are consistently gas-bounded and result-ignored.** The liquidation sweep, the seeder poke, the legacy buyback and the native gacha all reserve gas for the parent swap to finish and ignore failure, so none can revert or starve a user's trade. The `LIQ_GAS_RESERVE`/`LIQ_GAS_MIN` pattern is applied uniformly.
- **`_isExemptPlayer` closes a real hole.** Trusting `hookData` unconditionally would let anyone dodge the fee by tagging an exempt address; gating on `isOpener[sender]` fixes it, and the comment records the reasoning.
- **Perp bad-debt accounting distinguishes the long and short paths.** `_replenishPlv` and `_absorbPlvLoss` are genuinely different operations and the code implements both, with `plv` saturating at zero so an extreme gap cannot underflow.
- **`setVault`'s drain guard is strong.** The vault pointer cannot be re-pointed while depositor funds are staked, blocking the obvious owner-drain. (`skimInsurance` should be brought up to the same standard — **M-05**.)

- **The exit guarantee is real.** Arming an emergency action *forces* the redemption path open, so a custody move can never be preceded by a closed exit. This inverts the usual pause-then-move risk.
- **Relaunch cannot be bricked by its optional subsystems.** The perp force-close, the death checker, the fee router, the surtax/odds/curve policies and the badge mint are all `try/catch`'d or fallback-guarded. **C-02** is the notable exception — and it comes from a path that is *not* wrapped.
- `MigrationVesting` **verifies its own balance delta** (line 186) rather than trusting the registry's return value. That is exactly the defence **H-03** asks for, applied one layer higher.

8 The delivered test suite

Two new directories are added under `contracts/solidity/test/`.

8.1 `test/audit/` — exploit proofs

Nine tests, each corresponding to a finding. All pass; a passing test here means *the exploit works*.

Test	Finding	Proves
`PoC_PhantomRelaunchEth_BricksRelaunch`	C-01a	0.99 "ETH" minted from a non-ETH pool; `releaseRelaunchETH` reverts
`PoC_LegacyBufferTheft`	C-01b	5 ETH moves from hook to attacker
`PoC_OversizedNftSupplyBricksRelaunch`	C-02	`relaunch()` reverts forever; proposal not retired
`PoC_RegistryCannotCustodyTransfer`	H-01	The registry is not the collection's `deployer`
`PoC_DeployerIsFactoryNotRegistry`		
`PoC_CollectionAdminSettersUnreachable`		
`PoC_SecondCampaignNeverStreams`	H-02	91.5 % of gen-2 ledger A stranded
`PoC_ClaimFromReserveCapsInsteadOfReverting`	H-03	99 % shortfall, no revert
`PoC_RejectingTraderFreezesTheEngine`	H-04	Engine stuck on the dead generation
`PoC_FirstTokenStakerTakesTheWholePot`	H-05	Sniper takes 10/10 ETH; whale gets 0
`PoC_OwnerRepointsRegistryAndDrains`	M-01	Registry ACL moves; real registry locked out
`PoC_WinnerIsFrontRunnable`	M-02	One vote flips the winner
`PoC_HolderPicksTheBetterOfTwoRolls`	M-03	Two distinct rarity draws exist
`PoC_ForgedFrenTransferDoesNotBreakTheSpell`	M-06	Seller withdraws the buyer's 1 ETH

8.2 `test/invariants/` — invariants and fuzz

`CauldronSystemInvariants.t.sol`
: Stateful invariant suite against live v4. A single bounded actor drives buys, sells, seeder pokes, reserve donations, 1:1 migration, perp open/close/liquidate, time advancement and the permissionless relaunch; ghost variables track burned/received/donated totals. Eight `invariant_` functions cover I-1 through I-7 and I-9. To guarantee the invariants are not vacuous, the file also contains a *scripted lifecycle walk* that forces the full summon → stream → trade → death → relaunch → migrate path and re-asserts every invariant at each stage.

`LedgerInvariants.t.sol`
: Stateful invariants over `CollectionLedger` (accounting closure, entitlement never exceeding credit, well-formed `outstanding`) plus fuzz tests proving that a recycle never dilutes remaining holders and that a 2× buyback ratchets the floor up.

`VestingInvariants.t.sol`
: Escrow solvency, per-holder conservation, no over-release, and grant well-formedness under 128,000 randomised calls, plus fuzz tests for exact 1:1 escrow and the linear drip schedule.

`ReserveMathFuzz.t.sol`
: Pure fuzz over `ReserveLib` and `SeedLib`: reserve band well-formedness, the **orientation** property (band strictly below launch), round-*down* liquidity conversion, monotonicity, schedule monotonicity/cap, band single-sidedness, and taper weights summing to 10^{18}.

`FacetLayoutInvariant.t.sol`
: Runs the facet's compiled code against a third contract's storage image to prove it reads the declared slots; proves a direct call to the deployed facet is inert; fuzzes the exit guarantee.

8.3 Results

```
$ FOUNDRY_PROFILE=cauldron forge test
Ran 45 test suites: 245 tests passed, 0 failed, 0 skipped (245 total tests)

  test/invariants/CauldronSystemInvariants.t.sol .... 11 passed
  test/invariants/VestingInvariants.t.sol ........... 6 passed
  test/invariants/LedgerInvariants.t.sol ............ 3 passed (+3 fuzz)
  test/invariants/ReserveMathFuzz.t.sol ............. 8 passed
  test/invariants/FacetLayoutInvariant.t.sol ........ 5 passed
  test/audit/ (exploit proofs) ...................... 9 passed
  (pre-existing repository suite) ................... 218 passed
```

The pre-existing 218 tests were run before and after the additions and are unaffected; no production source file was modified by this audit.

9 Remediation plan

9.1 Before any value-bearing deployment

1. **C-01a + C-01b** — gate `_afterInitialize` on `sender == registry` and on `currency0 == address(0)`; pin the legacy buyback to the protocol's own `PoolKey`. *Two small diffs; both are unconditional and cheap to exploit.*
2. **C-02** — bound `nftSupply` in `propose` *and* clamp it in `relaunch`. *The clamp matters most: it survives a governor swap.*
3. **H-03** — add the short-delivery `require` to `migrateOne` and the two redemption paths.
4. **H-04** — convert the trader payout and keeper reward to the pull pattern. *Note **I-06**: extract a library first.*
5. **H-01** — give the collection an explicit controller address.
6. **H-02** — reset `complete` and `_basePlaced` in `startSeed`. *One line; the highest impact-to-effort ratio here.*
7. **H-05** — advance the yield watermark at zero shares.

9.2 Before mainnet

8. **M-01** — timelock the registry override and drain the reserve to the outgoing registry first.
9. **M-02** — add a voting deadline; only closed proposals may win.
10. **M-03 + M-04** — replace both deterministic fallbacks with re-anchoring.
11. **M-05** — derive the protected insurance minimum from live OI, and set a non-zero `insuranceFloor` at deploy.
12. **M-06** — notify the dividend on every transfer, not just genesis.
13. Set `EMERGENCY_DELAY` to a real value (the deploy default is **0**, making every "timelocked" break-glass action instant), and confirm the guardian is a multisig distinct from the emergency admin.

9.3 Hardening

Address L-01 through L-08 and the informational items; extract a `PerpOps` library to create EIP-170 headroom (**I-06**); and extend the repository's own `ProgressiveSeed.t.sol` to cover two consecutive progressive generations, which is the specific gap that let **H-02** through.

10 Second-pass findings: lifecycle denial of service

After the first round of fixes I re-attacked the protocol from a different angle: instead of asking *what can be drained*, I asked *what can be made to revert forever*. That framing is unusually productive here because `relaunch()` is permissionless, is the only path that carries value into the next generation, and — critically — retires the winning proposal *in the same transaction it might revert in*. Any reachable revert inside it is therefore not a temporary failure but a permanent, self-perpetuating one.

Five findings came out of that pass. Four are denial of service on the lifecycle; one is mark manipulation on the perp desk.

A-01 `CREATE2` token-address squatting permanently bricks the eternal machine Critical | Confirmed

Location. `cauldron/PoolOps.sol:415 deployToken` • `CauldronRegistry.sol:1170 _deployToken`

Mechanism. Every generation's ERC-20 was deployed at a fully deterministic address:

```
419 token = address(new CauldronToken{salt: bytes32(gen)}(
420             name, symbol, gen, address(this), totalSupply));
```

Every input is public *before* the transaction that would deploy it:

`salt`	the next generation number — `currentGeneration() + 1`
deployer	the registry (constant; `PoolOps` is delegatecalled)
`name`	`LaunchLib.displayName(spec.name)`
`symbol`	`spec.symbol`
`totalSupply`	`TOTAL_SUPPLY` (constant)

and `(name, symbol)` come from `governor.winner()` — a public view whose result is *frozen* once the leading proposal's voting window has closed. So anyone can compute the next generation's token address and deploy any contract there first.

Why it is permanent. CREATE2 into an occupied address fails; Solidity's `new{salt}` reverts on failure; and that revert propagates out of `relaunch()`. Because `governor.markConsumed(winId)` executed earlier *in the same transaction* (`CauldronRegistry.sol:679`), it is rolled back too — so the same proposal wins again, targeting the same squatted address, forever. This is the same self-perpetuating structure as **C-02**, reached by a completely different route.

Exploit path. Read `winner()`. Compute the address. Deploy anything there (a 5-byte stub suffices). Total cost: one transaction. Verified on a Sepolia fork in `test/attacks/A01_Create2Squat.t.sol`: before the fix, `relaunch()` reverted, `gov.consumedCount()` stayed `0`, and a retry 30 days later reverted identically.

Impact. Permanent, permissionless, single-transaction denial of the entire protocol lifecycle. No relaunch means no migration target for existing holders, no reserve for the next generation, LP locked in a dead zero-volume pool, and the perp engine stranded on a dead token. On the shipped deployment there is no recovery: `setGovernor` is `onlyOwner` and ownership is handed to the presale contract, which exposes no call to it.

Fix (applied). Use plain `CREATE`. The address then derives from `(registry, registry nonce)`, and only the registry can advance its own nonce — there is no address for an attacker to occupy. This is structural immunity rather than probabilistic hardening (a CREATE2 miner would need a 2^{160} preimage search to hit a specific address). Nothing depended on predictability: the `predictTokenAddress` helper had already been removed as having no callers.

```
- token = address(new CauldronToken{salt: bytes32(gen)}(
- name, symbol, gen, address(this), totalSupply));
+ token = address(new CauldronToken(name, symbol, gen, address(this), totalSupply));
```

Consumers must read the live address from `currentToken()` or the `CauldronSummoned` / `CauldronReborn` events rather than deriving it.

A-01b A dust reserve rounds to zero liquidity and reverts the seed High | Confirmed

Location. `cauldron/PoolOps.sol:479 _seedReserve`

Mechanism. `ReserveLib.liquidityForTokenOut` rounds *down* (correctly — see invariant R-3). A sufficiently small reserve tranche therefore maps to **zero** liquidity, and `MINT_POSITION` with zero liquidity reverts `CannotUpdateEmptyPosition` inside the PositionManager.

The guard above it only checked the token *amount*:

```
258 if (reserveTokens > 0) { // 2648 wei passes this...
259     (r.reserveTickLower, r.reserveTickUpper) = ReserveLib.reserveTicks(...);
260     r.reservePositionId = _seedReserve(pm, r.key, ..., reserveTokens, token);
261 } // ...but yields L = 0 -> revert
```

Reachability. This is not exotic. `newActive` is sized from the tokens rescued from the dead LP; when a generation dies having traded almost nothing, `newActive` absorbs nearly the entire supply and `newReserve = TOTAL_SUPPLY - newActive` is dust. I hit it on the first fork run of a plain summon-then-relaunch with no genesis bonus configured.

Impact. Same permanent-freeze structure as A-01 and C-02: the revert propagates out of `relaunch()` and rolls back `markConsumed`.

Fix (applied). Return `0` instead of minting an empty position. Every consumer already handles a zero reserve id — `claimFromReserve` and `addToReserve` no-op at zero liquidity, and `_removeLiquidity` skips a zero id — and it is the correct outcome: a dust reserve backs nothing, so there is nothing to place.

```
uint128 liquidity = ReserveLib.liquidityForTokenOut(tickLower, tickUpper, tokenAmount);
if (liquidity == 0) return 0; // <-- nothing to place; never mint an empty position
```

A-02 A stale `lastTick` lets an atomic round-trip poison the liquidation mark

High | Confirmed

Location. `cauldron/PerpEngine.sol:346 _writeObs • :365 twapTick`

Mechanism. Observation writes are throttled to one per `OBS_INTERVAL` (15s) so the ring cannot be flooded — a good defence. But the throttle used to bail out of the *entire* function:

```
346 uint32 dt = nowTs - lastObsTs;
347 if (dt < OBS_INTERVAL) return; // <-- bails out completely
348 tickCumulative += int56(lastTick) * int56(uint56(dt));
349 ...
350 lastTick = _currentTick(); // <-- so this is SKIPPED
```

leaving `lastTick` holding whatever the tick was at the last un-throttled write. `twapTick` then extrapolates the un-recorded tail with that value:

```
383 int56 cumNow = tickCumulative + int56(lastTick) * int56(uint56(nowTs - lastObsTs));
```

Exploit path. One atomic transaction:

1. **Crash** spot with a large sell. The hook's `afterSwap` sweep pokes the engine, a write lands, and `lastTick` freezes at the crashed tick.
2. **Restore** spot by buying back in the *same* transaction. `dt == 0`, so the old code returned early and `lastTick` stayed crashed.
3. **Wait.** The crashed tick is now credited for every second until the next write — up to a full `twapWindow` — even though spot never moved.

The attacker's only cost is the round-trip's fee and slippage. Verified in `test/attacks/A02_PerpAttacks.t.sol`: spot tick before and after the round-trip were identical (`172349`), yet a healthy 2× long became liquidatable and the attacker collected the keeper reward.

Impact. Defeats the protocol's stated guarantee that "liquidations are triggered off a time-weighted average tick … so a single-block flash-move can't farm liquidations." Solvent positions are force-closed and the attacker takes the keeper cut; the trader loses the liquidation penalty for nothing.

Fix (applied). Always integrate the elapsed interval and always refresh `lastTick`, so the tail is extrapolated at the tick genuinely in force. Ring *appends* keep their own throttle clock (`lastRingTs`, packed into the same storage slot), so flood-resistance is unchanged.

```
function _writeObs() internal {
    uint32 nowTs = uint32(block.timestamp);
    uint32 dt = nowTs - lastObsTs;
    if (dt > 0) { // ALWAYS integrate
        tickCumulative += int56(lastTick) * int56(uint56(dt));
        lastObsTs = nowTs;
    }
    if (nowTs - lastRingTs >= OBS_INTERVAL) { // appends stay throttled
        observations[obsIndex] = Observation(nowTs, tickCumulative);
        obsIndex = (obsIndex + 1) & OBS_MASK;
        lastRingTs = nowTs;
    }
    lastTick = _currentTick(); // ALWAYS refresh
}
```

A-03 The perp book can outgrow what one force-close clears, stranding the engine

High | Confirmed

Location. `cauldron/PerpEngine.sol:700 forceCloseAllDead • :725 syncGeneration`

Mechanism. `forceCloseAllDead` is bounded to 96 iterations for gas safety, and the registry drives it exactly *once*, at relaunch (`CauldronRegistry.sol:586`). Nothing bounded how many positions could exist. With a 0.003 ETH dust floor, 100 positions cost roughly 0.4 ETH to open.

Why it is unrecoverable. The leftovers cannot simply be closed afterwards. `syncGeneration` refuses to re-arm while `openCount != 0`, and after the rebirth `forceCloseDead` is unusable in two independent ways:

`_isDead()` now reads the *new* pool (alive), and even if it did not, `_settle` would swap the position's *old*-generation `size` against the *new* pool. So the engine is pinned to a dead generation with its entire short inventory denominated in a token nobody trades.

Fix (applied). Bound the book below the force-close bound, so "one relaunch drains the whole book" becomes structural:

```
uint256 public constant MAX_OPEN_POSITIONS = 64; // < FORCE_CLOSE_MAX (96)
uint256 internal constant FORCE_CLOSE_MAX = 96;

function _book(...) internal returns (uint256 id) {
    if (openCount >= MAX_OPEN_POSITIONS) revert OiCapped();
    ...
}
```

Deliberately a `constant`, not an owner-tunable: it underwrites a structural guarantee, so there should be no way to configure it into violation. The regression fills the book to exactly `MAX_OPEN_POSITIONS`, confirms the next open is refused, and confirms one `forceCloseAllDead` drains it.

Residual risk (accepted, documented). A capped book can be filled to block new perp opens. That griefing costs the 6.9 % open fee on every slot, carries real market risk, and exposes every slot to liquidation — and it only blocks the perp desk, never the protocol. That is a strictly better failure mode than permanently stranding the engine.

A-05 Recovered LP tokens can exceed `TOTAL_SUPPLY`, underflowing the relaunch — High | Confirmed

Location. `CauldronRegistry.sol:721`

Mechanism.

```
721 uint256 newReserve = TOTAL_SUPPLY - newActive; // reverts if newActive > TOTAL_SUPPLY
```

`newActive` is derived from `tokensFromLP` — what the dead pool actually returned. That is *not* bounded by the fixed supply, because `PoolOps.removeAll` uses `DECREASE_LIQUIDITY` + `TAKE_PAIR`, which collects accrued **fees** alongside principal. Uniswap v4 exposes `poolManager.donate()`, which credits arbitrary amounts to in-range liquidity providers — so anyone can inflate what the registry's position pays out at teardown.

Exploit path. Buy tokens on the open market, call `donate()` on the live pool. At the next relaunch the registry recovers more than `TOTAL_SUPPLY`, the subtraction underflows, and `relaunch()` reverts — rolling back `markConsumed` with it, permanently. The attacker pays market price for the donated tokens; the protocol is bricked forever. It is also reachable *by accident*, from anyone donating to "support" the pool.

Impact. Permanent lifecycle freeze, same structure as A-01 and C-02.

Fix (applied). Clamp. The newborn only ever holds `TOTAL_SUPPLY`, so an active tranche above it is meaningless; the surplus simply stays with the registry.

```
if (newActive > TOTAL_SUPPLY) newActive = TOTAL_SUPPLY;
uint256 newReserve = TOTAL_SUPPLY - newActive;
```

A-04 Funding is not a closed transfer, so it leaks LP principal — Medium | Confirmed

Location. `cauldron/PerpEngine.sol:797 (in _settle)`

Mechanism. The paying side was capped by its *own* residual; the receiving side was capped only by the *whole vault*:

```
795 if (fd > 0) { // crowded side PAYS
796     uint256 pay = uint256(fd);
797     if (pay > residual) pay = residual; // <-- capped by its OWN residual
798     plv += pay; residual -= pay;
799 } else if (fd < 0) { // underweight side RECEIVES
800     uint256 credit = uint256(-fd);
801     if (credit > plv) credit = plv; // <-- capped only by the WHOLE vault
802     plv -= credit; residual += credit;
803 }
```

Any position closing at a loss — and *every* liquidated long, where `repay = proceeds` leaves `residual == 0` — pays less than it owes, while receivers still draw in full. The difference came straight out of LP principal.

Fix (applied). Pay receivers from the `insuranceEth` buffer first, exactly as every other bad-debt path in the contract already does (`_replenishPlv`, `_absorbPlvLoss`), so an unmatched funding claim consumes the buffer that exists for it before touching depositor capital.

```
} else if (fd < 0) {
    uint256 credit = uint256(-fd);
    uint256 fromIns = credit < insuranceEth ? credit : insuranceEth;
    insuranceEth -= fromIns; // buffer absorbs it FIRST
    uint256 rest = credit - fromIns;
    if (rest > plv) rest = plv; // never overdraw the vault
    plv -= rest;
    residual += fromIns + rest;
}
```

Verified by a 257-run fuzz (`testFuzz_Invariant_A04_PlvNeverShrinksOnSolventRoundTrip`) that a solvent open-and-close round trip never shrinks the vault.

11 Gas optimisation

A gas pass was run after remediation, on the paths that execute most often: the V4 swap callbacks (every trade), and the perp engine's mark and settlement (every open, close and liquidation). Measurements are Foundry gas-report averages from the fork suite, before and after.

11.1 Applied optimisations

ID	Change	Why it pays
G-06	`twapTick`: linear ring scan → **binary search**	Observations are written in strictly increasing timestamp order into a wrapping ring, so the populated slots are already sorted from the oldest (`obsIndex` once wrapped). ≈5 SLOADs instead of an unconditional 32, on a path that runs on every open, close, liquidation and funding poke.
G-07	`OBS_CARDINALITY` wrap: modulo → **mask**	The ring is a power of two, so `& OBS_MASK` replaces `% OBS_CARDINALITY`. Cheaper in gas *and* bytecode.
G-09	Hash the `PoolId` **once per swap**	`key.toId()` is a `keccak256` over the 5-field `PoolKey`. The swap path hashed it three separate times (adoption gate, volume, fee). Now computed once in the callback and threaded down.
G-10	Volume ring: `uint256[24]` → `uint128[24]`	Two buckets share a slot, so the 24-bucket sum behind `isDead` costs 12 SLOADs instead of 24. A bucket holds one hour of ETH volume in wei; `uint128` caps at 3.4×10^{20} ETH, so the narrowing is unreachable — and the add saturates rather than wrapping.
G-05	Drop an `abi.encode`/`decode` round trip	The in-lock swap path marshalled a `SwapReq` to `bytes` and immediately decoded it. Only the `unlock` path needs the encoding; the in-lock path now passes the struct directly.
G-03	Remove the 2-arg `openLong` overload	A pure forwarder to the 3-arg form. Every caller (frontend, router, tests) already used the 3-arg signature.
G-04	Remove `liquidateInSwap` / `liquidateManyInSwap`	Dead: the hook has only ever called the hint-free `sweepLiquidations`. They also carried their own `_inLocked` reentrancy surface.
G-11	`MAX_OPEN_POSITIONS` as a `constant`	Removes a storage getter, its SLOAD, and a setter — and makes the A-03 guarantee non-misconfigurable.
G-01	`accumulatedTokenFees` + `withdrawTokenFees` removed	Dead surface (finding I-01).
G-02	`getVolume24h` loop `unchecked`	24 `uint128` addends cannot overflow a `uint256`.

11.2 Measured effect

Function	Before	After	Change
`CauldronHook.isDead` (median)	65,704	40,085	−39%
`PerpEngine.liquidate` (median)	152,104	93,167	−39%
`PerpEngine.poke` (avg)	89,357	63,108	−29%
`PerpEngine.openLong` (avg)	772,187	673,703	−13%
`PerpEngine.openShort` (avg)	773,263	671,775	−13%
`PerpEngine.forceCloseDead`	320,910	298,290	−7%

The `liquidate` and `isDead` improvements matter most: the former runs inside the in-swap sweep on *every* trade, where the gas budget is what decides whether an underwater position actually gets closed, and the latter gates every perp open as well as the relaunch check.
The `openLong`/`openShort` averages are dominated by the real pool swap, so the ≈13 % is the fixed overhead falling away rather than the swap getting cheaper.

11.3 Deliberately not done

- **Caching `_key()` in the perp engine.** It reads `registry.currentToken()` on every call, which is a cross-contract SLOAD. Caching it would save real gas but would introduce a staleness bug across a relaunch — exactly the class of defect A-03 and H-04 are about. Correctness wins.
- **Packing the `Position` struct.** `size` and `principal` are genuinely `uint256`-scale (token amounts at 10^{18} precision against a 777M supply), so narrowing them would risk truncation on the settlement path for a one-slot saving.
- **Extracting a `PerpOps` library for EIP-170 headroom.** Measured: the delegatecall stubs cost more bytecode than the extracted bodies saved (a net *1 byte* gain for the badge helper, and −866 bytes for the settlement tail). Removing dead code (G-03, G-04) reclaimed far more.

11.4 Remaining EIP-170 headroom

Contract	Runtime size	Margin
`CauldronHook`	24,518	58
`PerpEngine`	24,510	66
`CauldronRegistry`	23,620	956
`PoolOps`	20,012	4,564

This is the single most important operational note in the report. The hook and the engine sit within ≈60 bytes of the limit. Any future change to either — including a one-line security fix — will not compile. Before the next feature lands, extract a linked library from one of them along the `PoolOps` pattern the registry already uses. Do it while there is no incident pressure, not during one.

12 Remediation

Every finding in this report was fixed in this repository. The workflow for each was: demonstrate the exploit against the unpatched code, apply the fix, re-run the same exploit and confirm it fails, then rewrite the test as a permanent regression that asserts the fixed behaviour.

12.1 Regression coverage

File	Covers
`test/attacks/A01_Create2Squat.t.sol`	A-01 (address unpredictable; summon and relaunch both survive a squat)
`test/attacks/A02_PerpAttacks.t.sol`	A-02, A-03, A-04 (mark resists an atomic round-trip; a full book force-closes; funding never shrinks the vault)
`test/attacks/A05_ReserveFloorSeeder.t.sol`	A-05, plus migration exactness, seeder-fund isolation and facet freezing
`test/audit/AuditPoC.t.sol`	C-01a, C-01b, H-01, H-02, H-05, M-03
`test/audit/AuditPoC2.t.sol`	H-03, M-01, M-02, M-06, L-01
`test/audit/AuditPoC3.t.sol`	H-04 (both the single- and batch-close paths)
`test/audit/AuditPoC4.t.sol`	C-02 (registry clamp and governor rejection)
`test/invariants/`	The invariant register of §5

12.2 Deliberate non-fixes

Three items were considered and consciously left as they are. Each is a trade-off, not an oversight.

L-03 — `tx.origin` credit attribution.
ACCEPTED. The finding is correct: under ERC-4337 `tx.origin` is the bundler, so a relayer would accrue crystal credit for the operations it relays. But every alternative is worse for the case that dominates in practice. A user clicking *Buy* in any Uniswap UI or aggregator arrives with `sender` set to the *router*, so crediting `sender` would send the crystal — and the NFT it forges — to a router contract instead of the person who paid. Crediting the EOA that signed the transaction is the only attribution that reaches the buyer without a trusted tag. Smart-account users should route through a tagging opener, and `creditUntaggedSwaps` can disable the path entirely. Documented in the code.

I-04 — the vestigial `claimed` mapping.
DOCUMENTED. Removing it would shift the shared storage layout that the registry and the `RedemptionExt` facet must agree on slot-for-slot. The cost of re-verifying that layout outweighs the benefit of deleting an unused mapping.

I-05 — `liquidityForTokenOut` at extreme ticks.
DOCUMENTED. Unreachable for this token's economics (a Cauldron pool's launch tick is strongly positive, $\approx +204{,}000$; the overflow regime begins below roughly $-480{,}000$). The fuzz domain is bounded accordingly with an explanatory comment rather than hidden.

12.3 Deployment-script hardening

Two defaults were changed because they silently disabled protections the contracts implement correctly:

- `EMERGENCY_DELAY` defaulted to **0**, which made every "timelocked" break-glass action — `emergencyWithdrawLP`, `emergencySweep`, `migrateToSuccessor` — execute instantly, with no arm-and-wait window for holders to exit and none for the guardian to veto. The delay is *immutable* once constructed, so a zero could never be corrected. Now defaults to 48 hours.
- `insuranceFloor` was never set, leaving it **0**. That both disabled the opens circuit-breaker (`insuranceEth < insuranceFloor` is never true) and, before the M-05 fix, let `skimInsurance` drain the entire bad-debt buffer. Now armed explicitly at deploy.

12.4 Verification

```
$ FOUNDRY_PROFILE=cauldron forge test
Ran 48 test suites: 262 tests passed, 0 failed, 0 skipped (262 total tests)
```

That is the repository's original 218 tests — all still passing, unmodified except where a fix deliberately changed behaviour — plus 44 delivered here.

13 Complete function inventory

All **535** function declarations across the 31 in-scope files: 422 with bodies plus 113 interface declarations. Every entry was annotated by hand with its role in the protocol, the state it reads and writes, and the external calls it makes. Nothing is omitted.

Column key: **Vis.** = visibility (`external`, `public`, `internal`, `private`); **Mut.** = mutability (`view`, `pure`, `payable`); **Access** = modifiers other than `override`/`virtual`. Line numbers follow the declaration name.

`CauldronRegistry.sol` — contract `CauldronRegistry` (52 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`constructor` L122	inte	–	–	Wires poolManager/positionManager/hook into SHARED STORAGE (not immutables, so the facet reads them under delegatecall) and fixes emergencyAdmin/emergencyDelay. *State:* W poolManager, positionManager, hook, emergencyAdmin, emergencyDelay
`setRedemption Ext` L143	exte	–	only Owner	One-shot wiring of the delegatecall facet; rejects EOAs (an empty target would make every forwarded redemption a silent no-op). *State:* W redemptionExt *Calls:* ext.code.length
`setReserve Ceiling` L156	exte	–	only Owner	Per-iteration reserve ceiling (tick offset, default ≈ 69×), bounded [4000,138000]. *State:* W nextReserveCeilingOffset
`setSeeder` L174	exte	–	only Owner	Wires the progressive streamer and propagates it to the hook's in-swap nudge. *State:* W seeder *Calls:* hook.setSeeder
`setSeedWindow` L185	exte	–	only Owner	Launch window for the NEXT summon/relaunch; 0 = atomic. Bounded [60s, 7d]. *State:* W nextSeedWindow
`enchantFee` L194	exte	view	–	View: token fee a MOVED fren pays to re-enchant = floor × enchantFeeMultBps. *State:* R genesisReserveOutstanding, genesisShares, enchantFeeMultBps
`armEmergency` L217	exte	–	only Emergency	Announces a custody action on-chain; starts the timelock AND forces the redemption exit open. *State:* W emergencyReadyAt
`setGuardian` L225	exte	–	–	Sets the veto-only safety role (owner pre-handoff, emergencyAdmin after). *State:* W guardian
`vetoEmergency` L235	exte	–	–	Guardian-only cancel of an armed action. Can only block, never move funds. *State:* W emergencyReadyAt
`setRedemption Paused` L250	exte	–	only Emergency	Fast circuit-breaker on the OG redemption path (emergencyAdmin, not timelocked). *State:* W redemptionPaused
`setEnchantFee Mult` L259	exte	–	only Emergency	Tunes the re-enchant multiple, capped at 100×. *State:* W enchantFeeMultBps
`emergencyWithdraw LP` L266	exte	–	non Reentrant, only Emergency, time-locked	BREAK-GLASS: pulls a generation's active+reserve LP to the admin. Timelocked. *State:* R generationToken; via _removeLiquidity *Calls:* PoolOps.removeAll, ISeeder.withdrawAll, IERC20.transfer, admin.call
`emergencySweep` L279	exte	–	non Reentrant, only Emergency, time-locked	BREAK-GLASS: sweeps the registry's ETH or an ERC20 to the admin. Timelocked. *Calls:* IERC20.transfer, admin.call
`setSuccessor` L296	exte	–	only Emergency	Telegraphs the V2 pointer. Does not move custody. *State:* W successor
`setClaimGate` L305	exte	–	only Emergency	Routes instant 1:1 migration through the vesting escrow (0 = off). *State:* W claimGate

Function	Vis.	Mut.	Access	Role, state touched, external calls
migrateToSuccessor L320	exte	–	non Reentrant, only Emergency, time-locked	V2 HANDOFF: transfers the active+reserve position NFTs' OWNERSHIP (no LP teardown) plus loose token/ETH. Armed+timelocked+vetoable. *State:* R generationPositionId, generationReservePositionId, generationToken *Calls:* IERC721.transferFrom, IERC20.transfer, successor.call
receive L345	exte	paya	–	Accepts ETH (vault sweeps, hook releases, prime-buy funding).
setGovernor L352	exte	–	only Owner	Points relaunch at the proposal governor. *State:* W governor
setMinLifetime L357	exte	–	only Emergency	Grace period a newborn brew gets before it can be relaunched. *State:* W minLifetime
setFactory L362	exte	–	only Owner	Collection/vault factory (required before summon). *State:* W factory
setNftMaxSupply L367	exte	–	only Owner	Default per-brew NFT cap. *State:* W nftMaxSupply
setRoyalty L374	exte	–	only Owner	EIP-2981 receiver + rate ($\leq 10\%$) stamped into every brew's collection. *State:* W royaltyDividend, royaltyBps
setGenesisMetadata L382	exte	–	only Owner	Iteration-1 art source (renderer or baseURI). *State:* W genesisMode, genesisBaseURI, genesisRenderer
setGenesisBonus L398	exte	–	only Owner	Reserves a slice of gen-1 supply for the founding guild; pre-summon only, $\leq 30\%$. *State:* W mifrens, genesisBonusBps, genesisShares
setAirdropReserve L413	exte	–	only Owner	Carves an OG-airdrop tranche sent off-LP at summon; capped at 20% of supply. *State:* W airdropWallet, airdropReserve
setPrimeFunder L422	exte	–	only Owner	Authorises the prime-buy funder (survives the ownership handoff). *State:* W primeFunder
fundPrimeBuy L432	exte	paya	–	Funder pre-loads personal ETH for the genesis first-block market buy. *State:* W primeBuyEth
sweepPrimeBuy L438	exte	–	–	Funder reclaims un-spent prime-buy ETH. *State:* W primeBuyEth *Calls:* msg.sender.call
summon L459	exte	paya	non Reentrant, only Owner	GENESIS. Deploys gen-1 token (CREATE2), sizes the genesis bonus, sends the airdrop tranche, seeds the pool (progressive or green-candle), executes the prime buy, deploys the collection. *State:* W summoned, currentGeneration, currentToken, generationToken, genesisSharePerFren, genesisReserveOutstanding, lastSummonAt, primeBuyEth, _seedBuyUnlocked *Calls:* PoolOps.creatureFor/deployToken/createAndSeed*/primeBuy, IERC20.transfer, factory.deployBrew, hook.setCollection/setVault

Function	Vis.	Mut.	Access	Role, state touched, external calls
`relaunch` L552	exte	–	non Reentrant	PERMISSIONLESS REBIRTH. Checks death + grace + governance, force-closes perps, tears down the dead LP+seeder, burns recovered supply, drains tickets, closes the vault, pulls hook fees, sizes the new active/reserve split, deploys+seeds the newborn, re-wires the collection and the engine. *State:* W currentGeneration, currentToken, generationToken/Proposer/Parent, lastSummonAt, genesisReserveOutstanding, genesisPending *Calls:* hook.isDead/resolveTickets/relaunchETH/ releaseRelaunchETH/forceClosePerps/setActiveProposer/ setNftCurveFrom, governor.winner/markConsumed, IVaultClose.close, PoolOps.*, ISeeder.withdrawAll, CauldronToken.burn, factory.*
`_perpHousekeep` L727	priv	–	–	Best-effort perp relaunch housekeeping: force-close-all (pre-death) or syncGeneration (post-rebirth). try/catch so it can never brick the rebirth. *Calls:* hook.perpEngine, hook.forceClosePerps, IPerpSync. syncGeneration
`_deploy Collection` L745	priv	–	–	Deploys a brew's collection+vault via the factory and points the hook at them. *State:* W generationCollection, generationVault *Calls:* factory.deployBrew, hook.setCollection, hook. setVault
`_continueMi Frens` L794	priv	–	–	Iteration #2 special case: CONTINUES the canonical MiFrens collection (new vault only, hook becomes minter). *State:* W generationCollection, generationVault *Calls:* factory.deployVault, IMiFrensContinuable.setVault/ setMinter, hook.setCollection/setVault
`claimByBurn` L835	exte	–	–	1:1 MIGRATION. Burns the caller's previous-gen balance and releases the same amount from the out-of-range reserve. Deliberately NOT nonReentrant so the perp engine can migrate during relaunch. *State:* R claimGate, generationToken, generation reserve refs *Calls:* hook.perpEngine, PoolOps.migrateOne → CauldronToken.burn + PositionManager.modifyLiquidities
`enableAuto Migrate` L875	exte	paya	–	Opt into keeper-executed migration; free for fren holders, 0.069 ETH otherwise. *State:* W autoMigrate *Calls:* IERC721.balanceOf
`autoMigrate Batch` L894	exte	–	non Reentrant	Keeper batch migration for opted-in holders. Disabled while the vesting gate is on. *State:* R autoMigrate, claimGate *Calls:* PoolOps.autoMigrateBatch
`redeemOgFren` L937	exte	–	–	Thin DELEGATECALL forwarder to RedemptionExt. Must not be nonReentrant (the facet's guard uses the same slot). *Calls:* delegatecall redemptionExt
`buyTreasuryOg Fren` L943	exte	–	–	Forwarder to RedemptionExt. *Calls:* delegatecall redemptionExt
`donateToReserve` L949	exte	–	–	Forwarder to RedemptionExt. *Calls:* delegatecall redemptionExt
`materializeLegacy Reserve` L955	exte	–	–	Forwarder to RedemptionExt. *Calls:* delegatecall redemptionExt

Function	Vis.	Mut.	Access	Role, state touched, external calls
`_forwardToExt` L965	priv	–	–	Raw-assembly DELEGATECALL of the full calldata into the facet, bubbling returndata/reverts; rejects a zero target. *State:* R redemptionExt *Calls:* delegatecall
`setCollection Ledger` L987	exte	–	only Owner	Wires the legacy-floor cap table (0 = feature off). *State:* W collectionLedger
`_flushLegacyAt Relaunch` L1008	priv	–	–	At death, sweeps the hook's un-materialised buyback tokens, BURNS them, and credits the dying collection's ledger as a pure number. *State:* W genesisPending *Calls:* PoolOps.materializeLegacy
`recycleCollection NFT` L1022	exte	–	non Reentrant	Recycles a volume-collection NFT for its ledger floor, paid from the shared live reserve; NFT moves to the treasury. *State:* R collectionLedger, generationCollection, reserve refs *Calls:* PoolOps.recycleCollection → ledger.redeem + collection.custodyTransfer + reserve claim
`buyCollection NFT` L1040	exte	–	non Reentrant	Buys a treasury-held collection NFT at 2× floor; payment is added to the reserve and ratchets the floor. *State:* same as above *Calls:* PoolOps.buyCollection
`_remove Liquidity` L1063	priv	–	–	Removes BOTH positions of a retired generation and unwinds the seeder's mini-positions; returns (eth, tokens) recovered. *State:* R generationPoolKey/PositionId/ ReservePositionId, seeder *Calls:* PoolOps.removeAll, ISeeder.seeding/withdrawAll
`_deployToken` L1108	priv	–	–	CREATE2 token deploy delegated to PoolOps (keeps the 3.2 KB creation blob out of the registry). *Calls:* PoolOps.deployToken
`_createPoolAndSeed WithBuy` L1146	priv	–	–	ATOMIC seed: green-candle path. Arms _seedBuyUnlocked around the PoolManager re-entry. *State:* W _seedBuyUnlocked *Calls:* PoolOps.createAndSeedWithBuy
`_seed Generation` L1177	priv	–	–	Dispatches between the PROGRESSIVE handoff and the ATOMIC green-candle seed. *State:* R seeder, nextSeedWindow *Calls:* PoolOps.createAndSeedProgressive / _createPoolAndSeedWithBuy
`_recordSeed` L1198	priv	–	–	Persists poolId/key/positionIds/reserve ticks for a generation. *State:* W generationPoolId/PoolKey/PositionId/ ReservePositionId, reserveTickLower/Upper
`unlockCallback` L1215	exte	–	–	PoolManager unlock entry, accepted ONLY from the manager and ONLY while a seed/prime buy is armed. *State:* R poolManager, _seedBuyUnlocked *Calls:* PoolOps.executeBuy
`hasClaimed` L1225	exte	view	–	Legacy view over the (now unused) per-generation claimed map. *State:* R claimed

`CauldronHook.sol` — interface `IPerpEngineLiq` (3 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
liquidateInSwap L27	exte	–	–	Interface declaration — In-swap liquidation surface the hook calls from afterSwap.
liquidateManyIn Swap L28	exte	–	–	Interface declaration — In-swap liquidation surface the hook calls from afterSwap.
sweep Liquidations L29	exte	–	–	Interface declaration — In-swap liquidation surface the hook calls from afterSwap.

CauldronHook.sol — interface ICollectionLiquidator (1 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
setLiquidator Minter L34	exte	–	–	Interface declaration — Badge-minter wiring the hook auto-applies per brew.

CauldronHook.sol — interface ILegacyNote (1 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
noteLegacyBuy L40	exte	–	–	Interface declaration — Legacy buyback note (superseded by the sweep+credit pair).

CauldronHook.sol — interface IPerpForceClose (1 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
forceCloseAll Dead L45	exte	–	–	Interface declaration — Relaunch force-close surface.

CauldronHook.sol — interface IPerpFeeCredit (2 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
creditPerpFee L51	exte	paya	–	Interface declaration — Perp-fee credit surface (ETH side / token side).
creditPerpFee Token L52	exte	paya	–	Interface declaration — Perp-fee credit surface (ETH side / token side).

CauldronHook.sol — interface ISeederInSwap (1 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
pokeInSwap L59	exte	–	–	Interface declaration — The seeder's in-swap nudge.

CauldronHook.sol — contract CauldronHook (71 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
constructor L429	inte	–	–	BaseHook validates the mined address flags; sets deathThreshold, nftContract, treasury, explicit owner. *State:* W deathThreshold, nftContract, treasury *Calls:* Hooks.validateHookPermissions
getHook Permissions L445	publ	pure	–	Declares afterInitialize + before/afterSwap + both return-delta flags.

Function	Vis.	Mut.	Access	Role, state touched, external calls
`_afterInitialize` L473	inte	–	–	Marks ANY pool that names this hook as tracked and anchors its anti-snipe window. No allow-list (finding C-01). *State:* W trackedPools, _lastUpdateBlock, poolInitBlock
`_afterSwap` L493	inte	–	–	The heart: legacy buyback trigger, progressive poke, volume + crystal credit, hint-free perp liquidation sweep, native gacha, and the SELL-leg ETH fee via return delta. *State:* W volume buckets, cumulativeVolume, nftCredit, lifetimeVolumeOf, totalLifetimeVolume *Calls:* seeder.pokeInSwap, perpEngine.sweepLiquidations, self.nativeGachaStep, self.legacyBuyStep, poolManager.take, quest.call
`_maybeLegacyBuyback` L662	priv	–	–	Gas-bounded, result-ignored self-call that market-buys the token when the buffer is full. Uses the CURRENT swap's PoolKey (finding C-01). *State:* R legacyRegistry, legacyBuffer, legacyThreshold *Calls:* self.legacyBuyStep
`_maybePoke` L678	priv	–	–	Gas-bounded, result-ignored nudge of the progressive seeder from inside the swap (keeperless streaming). *State:* R seeder *Calls:* seeder.pokeInSwap
`legacyBuyStep` L694	exte	–	–	SELF-ONLY. Spends the buffered ETH on a nested exact-input buy, holds the tokens, and books legacyOwedToReserve. *State:* W legacyBuffer, legacyOwedToReserve, _inSelfBuy *Calls:* poolManager.swap/settle/take
`fundLegacyBuffer` L727	exte	paya	–	Permissionless top-up of the buyback buffer (the NFT-royalty entry point). *State:* W legacyBuffer
`sweepLegacyReserve` L734	exte	–	–	Registry-only handover of the held buyback tokens so they can be deposited into the reserve and credited atomically. *State:* W legacyOwedToReserve *Calls:* IERC20.balanceOf/transfer
`_beforeSwap` L750	inte	–	–	Charges the ETH fee on exact-input BUYS via a beforeSwapDelta. Correctly requires currency0 == address(0). *Calls:* _takeEthFee
`_routePerpFee` L794	priv	–	–	Perp-swap fee split: 30% to the OG dividend, 70% to the side-attributed staker pot; failures roll into relaunchETH. *State:* W relaunchETH *Calls:* guild.call, perpEngine.creditPerpFee/creditPerpFeeToken
`_routeEthFee` L812	priv	–	–	Organic fee split: proposer slice off the top (PULL), then guild/floor/relaunch with a pluggable IFeeRouter and a built-in fallback; legacyBps carved into the buyback buffer. *State:* W proposerOwed, legacyBuffer, relaunchETH *Calls:* feeRouter.route (try/catch), guild.call, vault.call
`snipeSurtaxBps` L900	publ	view	–	Anti-sniper surtax with a pluggable policy, clamped to MAX_SNIPE_BPS. *State:* R surtaxPolicy, snipeMaxBps, snipeWindowBlocks *Calls:* surtaxPolicy.surtaxBps (try/catch)
`_defaultSurtaxBps` L912	inte	view	–	Built-in linear decay plus a blockhash-derived jitter (predictable one block ahead — finding L-02). *State:* R poolInitBlock, snipeMaxBps, snipeWindowBlocks
`_takeEthFee` L943	priv	–	–	Takes the total ETH fee (base + surtax, clamped to 99%) out of the PoolManager and routes it. Takes key.currency0 WITHOUT checking it is ETH (finding C-01). *Calls:* poolManager.take, _routePerpFee/_routeEthFee, guild.call

Function	Vis.	Mut.	Access	Role, state touched, external calls
`setSnipeParams` L982	exte	–	only Owner	Owner-tunable anti-snipe window + peak surtax. *State:* W snipeWindowBlocks, snipeMaxBps
`_recordVolume` L992	priv	–	–	24-bucket rolling volume ring keyed on block number. *State:* W _volumeBuckets, _lastBucketIndex, _lastUpdateBlock
`getVolume24h` L1019	publ	view	–	Sums the ring; returns 0 once a full day of blocks has passed with no update. *State:* R _volumeBuckets, _lastUpdateBlock
`isDead` L1026	exte	view	–	The relaunch gate. Delegates to a pluggable IDeathChecker with a fallback to the built-in volume rule. *State:* R trackedPools, deathThreshold, deathChecker *Calls:* deathChecker.isDead (try/catch)
`_getCurrent Bucket` L1042	priv	view	–	block.number-derived hourly bucket index.
`_getHolderTax Rate` L1050	priv	view	–	Tiered tax from the NFT contract, keyed on the SWAP SENDER (the router, not the trader — finding L-04). *State:* R nftContract, defaultTaxBps *Calls:* INFTContract.getHolderTaxRate (try/catch)
`setDefaultTax Bps` L1062	exte	–	only Owner	Flat fee, capped at MAX_TAX_BPS (10%). *State:* W defaultTaxBps
`releaseRelaunch ETH` L1077	exte	–	–	Registry-only pull of the whole relaunch reserve. Reverts if the send fails — the DoS surface of finding C-01. *State:* W relaunchETH *Calls:* registry.call
`withdrawToken Fees` L1095	exte	–	–	Permissionless push of accumulated ERC20 fees to the treasury. DEAD: accumulatedTokenFees is never written (finding I-01). *State:* W accumulatedTokenFees *Calls:* IERC20.transfer
`setDeath Threshold` L1109	exte	–	only Owner	Owner-tunable 24h volume floor. *State:* W deathThreshold
`forceClose Perps` L1120	exte	–	–	Registry-only. Sets the transient relaunch-close flag so the engine's settlement swaps skip the fee/buyback, then force-closes every dead position. *State:* W _inRelaunchClose *Calls:* IPerpForceClose.forceCloseAllDead (try/catch)
`setLegacy Buyback` L1132	exte	–	–	Configures the live buyback (registry or hook owner). *State:* W legacyRegistry, legacyBps, legacyThreshold
`setDeath Checker` L1146	exte	–	–	Swaps the death RULE without an upgradeable proxy. *State:* W deathChecker
`setPolicies` L1156	exte	–	–	Swaps the surtax / odds / mint-curve policy modules. *State:* W surtaxPolicy, oddsPolicy, curvePolicy
`setFeeRouter` L1168	exte	–	–	Swaps the fee-split STRUCTURE (amounts only; the hook still does the sends). *State:* W feeRouter
`setNftContract` L1174	exte	–	only Owner	Points the tiered-tax lookup at an NFT contract. *State:* W nftContract
`setTreasury` L1178	exte	–	only Owner	ERC20 fee recipient. *State:* W treasury
`setRegistry` L1188	exte	–	only Owner	One-shot registry wiring, documented as "locked forever" — defeated by setRegistryOverride (finding M-01). *State:* W registry
`setRegistry Override` L1200	exte	–	only Owner	Owner may RE-POINT the registry at will (the V2 handoff hook). Also the drain path in finding M-01. *State:* W registry
`trackPool` L1205	exte	–	only Owner	Owner may mark an arbitrary pool tracked. *State:* W trackedPools, _lastUpdateBlock

Function	Vis.	Mut.	Access	Role, state touched, external calls
`setCollection` L1220	exte	–	–	Registry-only. Points the hook at the live collection, anchors the mint curve baseline, bumps the credit epoch, auto-wires badges. *State:* W collection, mintBaseline, creditEpoch *Calls:* ICauldronCollection.totalMinted, _wireLiquidator
`_wire Liquidator` L1238	priv	–	–	Best-effort wiring of the collection's Liquidatoor minter to the perp engine. *Calls:* ICollectionLiquidator.setLiquidatorMinter (try/catch)
`setVault` L1244	exte	–	–	Registry-only floor-vault pointer (0 under the unified-floor model). *State:* W vault
`setQuest` L1250	exte	–	–	Optional post-mintout engagement contract. *State:* W quest
`setSeeder` L1259	exte	–	–	Registry/owner wiring of the in-swap progressive nudge (0 disables it). *State:* W seeder
`setPerpEngine` L1267	exte	–	–	Registry/owner wiring of the hook-native perp engine; re-wires badge minting. *State:* W perpEngine *Calls:* _wireLiquidator
`setFloorBps` L1276	exte	–	only Owner	Share of the post-guild fee that backs the NFT floor. *State:* W floorBps
`setGuild` L1283	exte	–	–	Points the permanent OG-dividend sink. *State:* W guild
`setGuildBps` L1291	exte	–	only Owner	Owner-tunable guild share, capped at 5% — BELOW the deployed default of 15% (finding I-02). *State:* W guildBps
`setActive Proposer` L1299	exte	–	–	Registry pushes the winning proposal's author each relaunch. *State:* W activeProposer
`setProposerBps` L1306	exte	–	only Owner	Proposer incentive, capped at MAX_PROPOSER_BPS (5% of the fee). *State:* W proposerBps
`claimProposer Fees` L1314	exte	–	non Reentrant	PULL withdrawal of accrued proposer earnings (CEI + nonReentrant). *State:* W proposerOwed *Calls:* msg.sender.call
`setNftCurve` L1324	exte	–	only Owner	Owner-tunable rising mint curve (base, step). *State:* W volumePerNFT, nftPriceStep
`setCreditUntagged Swaps` L1330	exte	–	only Owner	Toggles crediting raw (non-router) swaps to tx.origin. *State:* W creditUntaggedSwaps
`setNftCurve From` L1337	exte	–	–	Registry-only flat curve taken from the winning BrewSpec's volumePerNFT. *State:* W volumePerNFT, nftPriceStep
`nftPriceAt` L1345	publ	view	–	Credit cost of the k-th crystal (pluggable curve policy, zero-guarded). *State:* R curvePolicy, volumePerNFT, nftPriceStep *Calls:* curvePolicy.priceAt (try/catch)
`creditOf` L1357	exte	view	–	Player's spendable credit in the current epoch. *State:* R nftCredit, creditEpoch
`_curvePos` L1365	inte	view	–	Curve position = minted + reserved − baseline (counting reserved stops concurrent commits sharing a slot). *State:* R collection, outstandingOf, mintBaseline *Calls:* ICauldronCollection.totalMinted
`crystalsReady` L1371	publ	view	–	How many crystals a player can open right now. *State:* R nftCredit *Calls:* _curvePos, nftPriceAt

Function	Vis.	Mut.	Access	Role, state touched, external calls
costOfNextCrystals L1385	publ	view	–	Total credit for the next N crystals at today's prices. *Calls:* nftPriceAt
oddsForPlay L1394	publ	view	–	Win probability from ETH play size, clamped to ODDS_HARD_CAP_BPS. *State:* R oddsPolicy, maxOddsBps, oddsFullVolumeWei *Calls:* oddsPolicy.oddsBps (try/catch)
outstandingTickets L1408	exte	view	–	Unresolved crystals across all players. *State:* R outstandingCrystals
mintedOut L1413	exte	view	–	Whether the live collection is sold out. *State:* R collection *Calls:* ICauldronCollection.totalMinted/maxSupply
progress L1423	exte	view	–	UI view: banked credit, next threshold, crystals ready. *State:* R nftCredit *Calls:* nftPriceAt
commitCrystals L1450	exte	–	non Reentrant	Opener-gated commit: spends credit and enqueues a FIFO lottery batch whose outcome is rolled later from the commit block's hash. *State:* W nftCredit, committedOf, pendingOf, outstandingCrystals, outstandingOf, batches *Calls:* ICauldronCollection.totalMinted/maxSupply
_commitCrystals L1462	inte	–	–	Shared commit body (router + native in-swap paths); never reverts when sold out. *State:* same as above
resolveTickets L1518	publ	–	non Reentrant	Permissionless FIFO resolution; a win mints the batch's collection, a miss builds the pity counter. *State:* W batchCursor, batches[].resolved, pendingOf, outstandingCrystals, missStreak, opened *Calls:* ICauldronCollection.mint
_resolveTickets L1530	inte	–	–	Resolution body. Uses blockhash(commitBlock) with a DETERMINISTIC fallback after 256 blocks (finding M-04). *State:* same as above
nativeGachaStep L1582	exte	–	non Reentrant	Self-only in-swap gacha for raw (non-router) buys: commit + resolve, gas-bounded and result-ignored. *Calls:* _commitCrystals, _resolveTickets
setOpener L1591	exte	–	–	Authorises a gacha router (and, in production, the registry) to open crystals + carry honest hookData. *State:* W isOpener
setTaxExempt L1601	exte	–	–	Flags a fee-exempt player (the launch snipe wallet and the registry's own reseed buy). *State:* W taxExempt
_isExemptPlayer L1615	priv	view	–	Exemption is honoured only when the SENDER is a trusted opener, closing the hookData-spoof hole. *State:* R isOpener, taxExempt
setOddsParams L1623	exte	–	only Owner	Odds curve size + pity threshold. *State:* W oddsFullVolumeWei, pityThreshold
setMaxOdds L1630	exte	–	only Owner	Max win chance from bet size, hard-capped. *State:* W maxOddsBps
setWeights L1636	exte	–	only Owner	Buy/sell credit weighting, capped at 3×. *State:* W buyWeightBps, sellWeightBps
receive L1642	exte	paya	–	Accepts raw ETH (fee takes land here); NOT counted into any tracked bucket.

CauldronToken.sol — contract CauldronToken (2 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`constructor` L36	inte	–	–	Mints the ENTIRE fixed supply once, to the registry. No mint() exists; supply can only ever shrink. *State:* W generation, birthBlock, registry, balances *Calls:* ERC20._mint
`burn` L58	exte	–	only Registry	Registry-only burn — used for dead-LP recovery, 1:1 migration, and the legacy flush. *State:* W totalSupply, balances

`cauldron/CauldronBase.sol` — interface `ICauldronFactory` (2 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`deployBrew` L25	exte	–	–	Interface declaration — Factory surface the registry drives; declared in CauldronBase so registry and facet see identical types.
`deployVault` L26	exte	–	–	Interface declaration — Factory surface the registry drives; declared in CauldronBase so registry and facet see identical types.

`cauldron/CauldronBase.sol` — interface `IMiFrensContinuable` (5 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`setMinter` L32	exte	–	–	Interface declaration — The canonical MiFrens surface for the iteration-#2 continuation and the OG recycle.
`setVault` L33	exte	–	–	Interface declaration — The canonical MiFrens surface for the iteration-#2 continuation and the OG recycle.
`totalMinted` L34	exte	view	–	Interface declaration — The canonical MiFrens surface for the iteration-#2 continuation and the OG recycle.
`custodyTransfer` L38	exte	–	–	Interface declaration — The canonical MiFrens surface for the iteration-#2 continuation and the OG recycle.
`everMoved` L41	exte	view	–	Interface declaration — The canonical MiFrens surface for the iteration-#2 continuation and the OG recycle.

`cauldron/CauldronBase.sol` — interface `IVaultClose` (1 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`close` L45	exte	–	–	Interface declaration — Vault close/sweep at relaunch.

`cauldron/CauldronBase.sol` — interface `IPerpSync` (1 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`syncGeneration` L51	exte	–	–	Interface declaration — Perp relaunch housekeeping.

`cauldron/CauldronBase.sol` — interface `ICollectionLedger` (1 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`totalEntitled` L55	exte	view	–	Interface declaration — Legacy cap-table read the registry sizes the new reserve against.

cauldron/CauldronBase.sol — interface IPositionManager (3 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
modifyLiquidities L63	exte	paya	–	Interface declaration — Minimal V4 PositionManager surface (declared locally to avoid permit2's solc pin).
nextTokenId L64	exte	view	–	Interface declaration — Minimal V4 PositionManager surface (declared locally to avoid permit2's solc pin).
getPositionLiquidity L65	exte	view	–	Interface declaration — Minimal V4 PositionManager surface (declared locally to avoid permit2's solc pin).

cauldron/CauldronBase.sol — abstract contract CauldronBase (3 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
floorPerFren L282	publ	view	–	LIVE per-fren redemption value = genesisReserveOutstanding / genesisShares. Shared by the registry AND the facet. *State:* R genesisReserveOutstanding, genesisShares
_redeemBlocked L292	inte	view	–	THE EXIT GUARANTEE: blocked only while the breaker is on AND nothing is armed. *State:* R redemptionPaused, emergencyReadyAt
constructor L296	inte	–	–	Ownable(msg.sender). Declares the whole shared storage layout the delegatecall pair depends on.

cauldron/CauldronCollection.sol — contract CauldronCollection (20 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
constructor L113	inte	–	–	Fixes the minter (the hook), the DEPLOYER (the FACTORY — finding H-01), maxSupply (must stay below the badge id range), metadata and royalty. *State:* W minter, deployer, maxSupply, mode, renderer, _baseTokenURI, royalty *Calls:* ERC2981._setDefaultRoyalty
_update L146	inte	–	–	ERC-721C: routes every mint/transfer/burn through the transfer validator when one is set. *Calls:* ITransferValidator.validateTransfer
getTransferValidator L159	exte	view	–	ERC-721C discovery.
getTransferValidationFunction L164	exte	pure	–	ERC-721C discovery.
setTransferValidator L169	exte	–	–	Deployer-gated — unreachable in production (finding H-01). *State:* W transferValidator
supportsInterface L176	publ	view	–	ERC165 over ERC721 + ERC2981 + ICreatorToken.
mint L184	exte	–	–	Minter-only art mint; records the mint block for the commit-reveal rarity roll. *State:* W totalMinted, mintBlockOf *Calls:* ERC721._mint
reveal L196	exte	–	–	Owner-only rarity reveal from blockhash(mintBlock), with a deterministic fallback after 256 blocks (finding M-03). *State:* W rarityOf, revealed
_rollRarity L213	priv	view	–	Cumulative-bps tier roll. *State:* R rarityCumBps

Function	Vis.	Mut.	Access	Role, state touched, external calls
`setVault` L222	exte	–	–	One-shot vault wiring (the factory does this at deploy). *State:* W vault
`setRoyalty` L232	exte	–	–	Deployer-gated royalty re-point (the factory does this at deploy). *Calls:* _setDefaultRoyalty
`setLiquidator Minter` L242	exte	–	–	Deployer OR minter (so the hook can auto-wire badges each relaunch). *State:* W liquidatorMinter
`setMetadata` L252	exte	–	–	Deployer-gated art repoint — unreachable in production (finding H-01). *State:* W mode, renderer, _baseTokenURI
`setLiquidator URI` L264	exte	–	–	Deployer-gated badge metadata base — unreachable in production. *State:* W liquidatorURI
`mintLiquidator` L272	exte	–	–	Engine-only trophy mint in a separate uncapped id range. *State:* W liquidatorMinted, isLiquidatoor *Calls:* ERC721._mint
`liquidatoor Trait` L281	exte	view	–	Marketplace helper.
`burnFromVault` L286	exte	–	–	Vault-only burn on ETH-floor redemption. *Calls:* ERC721._burn
`custodyTransfer` L296	exte	–	–	Approval-free move for the legacy recycle. Gated on 'deployer', which is the FACTORY — so the registry can never call it (finding H-01). *Calls:* ERC721._transfer
`tokenURI` L302	publ	view	–	Badge / unrevealed / renderer / rarity-partitioned baseURI resolution. *Calls:* ICollectionRenderer.tokenURI
`setRarityOdds` L324	exte	–	–	Deployer-gated, pre-first-mint odds config — unreachable in production. *State:* W rarityCumBps

`cauldron/CauldronFactory.sol` — contract `CauldronFactory` (2 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`deployBrew` L33	exte	–	–	Deploys collection + vault + royalty router and wires them. The FACTORY becomes the collection's 'deployer' (finding H-01). *Calls:* new CauldronCollection/CauldronVault/ RoyaltyRouter, col.setVault/setRoyalty
`deployVault` L59	exte	–	–	Deploys a fresh per-iteration vault for an EXISTING collection (the MiFrens continuation). *Calls:* new CauldronVault

`cauldron/CauldronGachaRouter.sol` — interface `ICauldronHookGacha` (5 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`commitCrystals` L15	exte	–	–	Interface declaration — Hook gacha surface the router drives.
`resolveTickets` L16	exte	–	–	Interface declaration — Hook gacha surface the router drives.
`crystalsReady` L17	exte	view	–	Interface declaration — Hook gacha surface the router drives.
`costOfNext Crystals` L18	exte	view	–	Interface declaration — Hook gacha surface the router drives.

Function	Vis.	Mut.	Access	Role, state touched, external calls
`buyWeightBps` L19	exte	view	–	Interface declaration — Hook gacha surface the router drives.

`cauldron/CauldronGachaRouter.sol` — interface `IRegistryCurrent` (1 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`currentToken` L23	exte	view	–	Interface declaration — Live-token read.

`cauldron/CauldronGachaRouter.sol` — contract `CauldronGachaRouter` (16 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`constructor` L91	inte	–	–	Binds the PoolManager, the hook, the registry and the owner.
`_key` L101	inte	view	–	Rebuilds the live PoolKey from registry.currentToken(). *Calls:* registry.currentToken
`play` L113	exte	paya	non Reentrant	One-click buy and/or sell, tagged with the PLAYER so the hook credits them, then commit + resolve crystals. *Calls:* poolManager.unlock, hook.commitCrystals/ resolveTickets
`playLiq` L128	exte	paya	non Reentrant	Same as play but carries perp liquidation hints in hookData. *Calls:* _play
`_play` L137	inte	–	–	Body: pulls the sell input, runs the unlock, enforces slippage, commits+resolves (effects), then pays out (interactions). *Calls:* _safeTransferFrom, poolManager.unlock, hook. commitCrystals/resolveTickets, msg.sender.call
`openReady` L189	exte	–	non Reentrant	Opens crystals from ALREADY-earned credit; derives an honest play size from the credit spent. *Calls:* hook.crystalsReady/costOfNextCrystals/ buyWeightBps/commitCrystals/resolveTickets
`playChurn` L207	exte	paya	non Reentrant	Volume amplifier: up to 10 buy→sell legs in one tx, each credited to the player (each leg pays the hook fee). *Calls:* poolManager.unlock, hook.commitCrystals/ resolveTickets
`unlockCallback` L236	exte	–	–	PoolManager-only dispatcher for the play and churn bodies. *Calls:* _churn / inline swap legs
`_churn` L286	priv	–	–	The churn loop: alternating exact-input swaps, settling each leg. *Calls:* poolManager.swap, _settle, _take
`_limit` L333	priv	pure	–	Directional sqrtPrice limits.
`_settle` L337	priv	–	–	Native settle-with-value or sync+transfer+settle for the ERC20 leg. *Calls:* poolManager.sync/settle
`_take` L347	priv	–	–	poolManager.take wrapper. *Calls:* poolManager.take
`_safeTransfer` L351	priv	–	–	Return-value-tolerant ERC20 transfer. *Calls:* token.call
`_safeTransfer From` L357	priv	–	–	Return-value-tolerant ERC20 transferFrom. *Calls:* token.call
`rescueETH` L363	exte	–	only Owner	Owner sweep of stray ETH (the router holds nothing between transactions). *Calls:* to.call

Function	Vis.	Mut.	Access	Role, state touched, external calls
`receive` L368	exte	paya	–	Accepts ETH (sell proceeds land here transiently).

`cauldron/CauldronGovernor.sol` — contract `CauldronGovernor` (11 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`constructor` L86	inte	–	–	Binds the MiFrens checkpointed-voting electorate. *State:* W mifrens
`setRegistry` L94	exte	–	only Owner	One-shot, owner-gated wiring of the registry allowed to consume winners. *State:* W registry
`propose` L123	exte	–	–	Guild-gated proposal of the next brew (name, ticker, metadata, NFT supply, mint-out target). NO bound on nftSupply (finding C-02). *State:* W proposalCount, _proposals *Calls:* mifrens.getVotes
`displayName` L167	exte	view	–	"⟨name⟩ by Magic Internet Frens".
`vote` L181	exte	–	–	One vote per address per proposal, weighted by CHECKPOINTED power at the proposal's snapshot block. *State:* W hasVoted, _proposals[].votes, _leaderId, _leaderVotes *Calls:* mifrens.getPastVotes
`winner` L211	exte	view	–	Returns the LIVE leader — there is no voting deadline, so it is front-runnable (finding M-02). *Calls:* _bestUnconsumed
`hasProposals` L229	exte	view	–	Whether any unconsumed proposal has votes. *Calls:* _bestUnconsumed
`markConsumed` L234	exte	–	–	Registry-only retirement of a summoned proposal; recomputes the leader lazily. *State:* W _proposals[].consumed, _leaderId, _leaderVotes
`getProposal` L252	exte	view	–	Full proposal record.
`_bestUnconsumed` L259	priv	view	–	Cached leader if still valid, else a full recompute.
`_recompute Leader` L267	priv	view	–	O(n) scan for the highest-voted unconsumed proposal.

`cauldron/CauldronSeeder.sol` — contract `CauldronSeeder` (18 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`constructor` L110	inte	–	–	Binds the registry, the (unused) PositionManager and the PoolManager. *State:* W registry, positionManager, poolManager
`receive` L116	exte	paya	–	Accepts ETH (ledger-A funding and pool takes).
`startSeed` L125	exte	paya	lock, only Registry	Registry-only campaign start: pulls ledger-A tokens, receives ledger-A ETH, places the base + floor slice. Does NOT reset 'complete'/'_basePlaced' (finding H-02). *State:* W _key, token, gen, startTs, window, seedFloorWad, ethTotal, tokenTotal, minStepWad, baseWad, _spacing, _bandWidth, seeding, placedWad *Calls:* IERC20.transferFrom, poolManager.unlock
`poke` L163	exte	–	lock	Permissionless standalone nudge (self-unlock). Un-accelerable: the target is a pure function of elapsed time. *State:* W placedWad, complete *Calls:* poolManager.unlock, _advance

Function	Vis.	Mut.	Access	Role, state touched, external calls
`pokeInSwap` L173	exte	–	lock	Hook-only in-swap nudge: places DIRECTLY because the caller already holds the unlock. *State:* W placedWad, complete *Calls:* _placeStep, _advance
`_pendingStep` L182	priv	view	–	The throttled step to deploy now; returns 0 when not seeding, complete, or below minStepWad. *State:* R seeding, complete, placedWad, startTs, window *Calls:* SeedLib.deployedTargetWad
`_advance` L193	priv	–	–	Snaps placedWad to the schedule target and flips ‘complete‘ at 100%. *State:* W placedWad, complete *Calls:* poolManager.getSlot0
`unlockCallback` L207	exte	–	–	PoolManager-only dispatcher for the PLACE and WITHDRAW bodies. *Calls:* _placeStep / _teardown
`_placeStep` L226	priv	–	–	Places one ASK (token, below spot) + one BID (ETH, above spot) band via CORE modifyLiquidity, laying the two-sided base on the first placement. *State:* W ranges, _basePlaced *Calls:* poolManager.getSlot0/modifyLiquidity, _settle, _track
`_teardown` L269	priv	–	–	Removes every tracked range and forwards all recovered + un-streamed funds to the registry. *State:* W _lastEthOut, _lastTokenOut *Calls:* poolManager.getPositionInfo/modifyLiquidity, IERC20.transfer, to.call
`_placeBase` L303	priv	–	–	Lays baseWad of ledger A as ONE two-sided full-range position so the book is continuous and perps have spot-straddling depth. *State:* W ranges *Calls:* poolManager.getSlot0/modifyLiquidity
`_settle` L325	priv	–	–	Pays owed currencies and takes owed-to-us ones; settles currency1 first so the synced-currency slot is reset before the native settle. *Calls:* poolManager.sync/settle/take, IERC20.transfer
`_track` L344	priv	–	–	Records a distinct (lo,hi) range once; reverts BandCap at MAX_RANGES = 64 so teardown gas stays bounded. *State:* W ranges
`withdrawAll` L360	exte	–	lock, only Registry	Registry-only teardown at death; ends the campaign and clears the range set. *State:* W ranges, seeding, complete *Calls:* poolManager.unlock
`rescue` L376	exte	–	lock, only Registry	Registry-only escape hatch for loose ledger-A funds. UNREACHABLE: the registry exposes no caller (finding I-03). *State:* W seeding *Calls:* IERC20.transfer, to.call
`deployedWad` L387	exte	view	–	Fraction of the stream budget deployed so far. *State:* R placedWad
`rangeCount` L388	exte	view	–	Number of distinct tracked ranges. *State:* R ranges
`isComplete` L389	exte	view	–	Whether the campaign has finished streaming. *State:* R complete

`cauldron/CauldronVault.sol` — interface `IBurnableCollection` (3 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`ownerOf` L7	exte	view	–	Interface declaration — Collection surface the ETH floor vault burns through.
`totalMinted` L8	exte	view	–	Interface declaration — Collection surface the ETH floor vault burns through.
`burnFromVault` L9	exte	–	–	Interface declaration — Collection surface the ETH floor vault burns through.

`cauldron/CauldronVault.sol` — contract `CauldronVault` (6 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`outstanding` L54	publ	view	–	NFTs this vault backs = eligible minted (above floorOffset) − redeemed. *State:* R redeemed, floorOffset *Calls:* collection.totalMinted
`constructor` L64	inte	–	–	Binds the collection, the registry and the genesis floorOffset. *State:* W collection, registry, floorOffset
`receive` L71	exte	paya	–	Accepts fee ETH (unused under the unified-floor model, where hook.setVault(0) is wired).
`floorPerNFT` L76	publ	view	–	balance / outstanding. *Calls:* outstanding
`redeem` L84	exte	–	non Reentrant	Burns an eligible NFT for its equal share of the pooled ETH. Always reverts under the unified model (no ETH ever arrives). *State:* W redeemed *Calls:* collection.ownerOf/burnFromVault, msg.sender.call
`close` L103	exte	–	non Reentrant	Registry-only: stops redemption and sweeps the remaining ETH into the next launch. *State:* W closed *Calls:* registry.call

`cauldron/CollectionLedger.sol` — contract `CollectionLedger` (7 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`constructor` L72	inte	–	–	Binds the sole mutator (the registry). *State:* W registry
`outstanding` L86	publ	view	–	Entitled NFT count = supply (live mint count or frozen snapshot) − retired. *State:* R crystallized, frozenSupply, retired
`floorPerNFT` L94	publ	view	–	Tokens one NFT of a generation can redeem right now. *State:* R entitledTokens
`credit` L106	exte	–	only Registry	Registry-only: grows a collection's entitlement (live buyback / royalty inflow). *State:* W entitledTokens, totalEntitled
`redeem` L117	exte	–	only Registry	Registry-only: pays one NFT its floor (round DOWN), retires it, debits the pot. *State:* W entitledTokens, retired, totalEntitled
`buyback` L130	exte	–	only Registry	Registry-only: credits a 2×-floor resale and un-retires the NFT (floor ratchet). *State:* W entitledTokens, retired, totalEntitled
`crystallize` L143	exte	–	only Registry	Registry-only one-time freeze of a dead collection's supply plus a final entitlement fold. *State:* W crystallized, frozenSupply, entitledTokens, totalEntitled

`cauldron/DefaultFeeRouter.sol` — **contract** `DefaultFeeRouter` (1 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`route` L21	exte	pure	–	Reference IFeeRouter reproducing the hook's built-in split; pure, owner-less, custody-free.

`cauldron/ICauldron.sol` — **interface** `ICollectionRenderer` (1 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`tokenURI` L12	exte	view	–	Interface declaration — On-chain art renderer surface.

`cauldron/ICauldron.sol` — **library** `LaunchLib` (1 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`displayName` L37	inte	pure	–	Hardcoded on-chain branding: "⟨name⟩ by Magic Internet Frens".

`cauldron/ICauldron.sol` — **interface** `ICauldronGovernor` (3 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`winner` L44	exte	view	–	Interface declaration — Governor surface the registry consumes on relaunch.
`markConsumed` L45	exte	–	–	Interface declaration — Governor surface the registry consumes on relaunch.
`hasProposals` L46	exte	view	–	Interface declaration — Governor surface the registry consumes on relaunch.

`cauldron/ICauldron.sol` — **interface** `ICauldronCollection` (3 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`mint` L51	exte	–	–	Interface declaration — Mint surface the volume hook drives.
`totalMinted` L52	exte	view	–	Interface declaration — Mint surface the volume hook drives.
`maxSupply` L53	exte	view	–	Interface declaration — Mint surface the volume hook drives.

`cauldron/ICreatorToken.sol` — **interface** `ITransferValidator` (1 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`validateTransfer` L8	exte	view	–	Interface declaration — ERC-721C validator the collections defer to.

`cauldron/ICreatorToken.sol` — **interface** `ICreatorToken` (3 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`getTransferValidator` L16	exte	view	–	Interface declaration — ERC-721C discovery surface markets recognise.
`getTransferValidationFunction` L17	exte	view	–	Interface declaration — ERC-721C discovery surface markets recognise.
`setTransferValidator` L18	exte	–	–	Interface declaration — ERC-721C discovery surface markets recognise.

`cauldron/IDeathChecker.sol` — interface `IDeathChecker` (1 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`isDead` L27	exte	view	–	Interface declaration — Pluggable death rule.

`cauldron/ILiquidatorMintable.sol` — interface `ILiquidatorMintable` (1 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`mintLiquidator` L11	exte	–	–	Interface declaration — Liquidatoor badge mint surface.

`cauldron/IPolicies.sol` — interface `ISurtaxPolicy` (1 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`surtaxBps` L25	exte	view	–	Interface declaration — Pluggable anti-snipe surtax curve.

`cauldron/IPolicies.sol` — interface `IOddsPolicy` (1 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`oddsBps` L37	exte	view	–	Interface declaration — Pluggable gacha odds curve.

`cauldron/IPolicies.sol` — interface `ICurvePolicy` (1 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`priceAt` L49	exte	view	–	Interface declaration — Pluggable mint-cost curve.

`cauldron/IPolicies.sol` — interface `IFeeRouter` (1 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`route` L70	exte	view	–	Interface declaration — Pluggable ETH fee-split STRUCTURE (amounts only; the hook keeps custody).

`cauldron/ISeeder.sol` — interface `ISeeder` (5 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`startSeed` L30	exte	paya	–	Interface declaration — Progressive seeder surface shared by PoolOps and CauldronSeeder.

Function	Vis.	Mut.	Access	Role, state touched, external calls
`poke` L31	exte	–	–	Interface declaration — Progressive seeder surface shared by PoolOps and CauldronSeeder.
`withdrawAll` L32	exte	–	–	Interface declaration — Progressive seeder surface shared by PoolOps and CauldronSeeder.
`isComplete` L33	exte	view	–	Interface declaration — Progressive seeder surface shared by PoolOps and CauldronSeeder.
`seeding` L34	exte	view	–	Interface declaration — Progressive seeder surface shared by PoolOps and CauldronSeeder.

`cauldron/LaunchSniper.sol` — interface `IMiFrensGenesisFinalize` (2 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`finalize` L8	exte	–	–	Interface declaration — Presale ignition surface.
`soldOut` L9	exte	view	–	Interface declaration — Presale ignition surface.

`cauldron/LaunchSniper.sol` — interface `IRegistryCurrent` (1 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`currentToken` L13	exte	view	–	Interface declaration — Live-token read.

`cauldron/LaunchSniper.sol` — interface `IGachaPlay` (1 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`play` L17	exte	paya	–	Interface declaration — Router play surface.

`cauldron/LaunchSniper.sol` — contract `LaunchSniper` (4 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`constructor` L47	inte	–	–	Owner binding.
`launch` L58	exte	paya	only Owner	ATOMIC ignition + funding buy: finalize the presale (summons gen-1 and seeds the pool) then immediately buy through the gacha router and forward the tokens to the airdrop wallet. Nothing can execute in between. *Calls:* presale.soldOut/finalize, registry.currentToken, gachaRouter.play, IERC20.balanceOf/transfer
`sweep` L86	exte	–	only Owner	Owner recovery of stray ETH/tokens. *Calls:* owner.call, IERC20.transfer
`receive` L95	exte	paya	–	Accepts refunds from the router.

`cauldron/MiFrensDividend.sol` — interface `IMiFrensShares` (4 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`ownerOf` L8	exte	view	–	Interface declaration — MiFrens reads the dividend needs.
`GENESIS_SUPPLY` L9	exte	view	–	Interface declaration — MiFrens reads the dividend needs.
`MAX_SUPPLY` L10	exte	view	–	Interface declaration — MiFrens reads the dividend needs.
`everMoved` L12	exte	view	–	Interface declaration — MiFrens reads the dividend needs.

cauldron/MiFrensDividend.sol — interface IReserveRegistry (3 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
enchantFee L18	exte	view	–	Interface declaration — Registry reads the dividend needs to price + route the re-enchant fee.
currentToken L19	exte	view	–	Interface declaration — Registry reads the dividend needs to price + route the re-enchant fee.
donateToReserve L20	exte	–	–	Interface declaration — Registry reads the dividend needs to price + route the re-enchant fee.

cauldron/MiFrensDividend.sol — contract MiFrensDividend (14 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
constructor L106	inte	–	–	Reads GENESIS_SUPPLY / MAX_SUPPLY off the collection and fixes the treasury sink. *State:* W SHARES, MAX_TOKEN, treasury *Calls:* IMiFrensShares.GENESIS_SUPPLY/ MAX_SUPPLY
setRegistry L120	exte	–	–	One-shot, treasury-gated wiring so a moved fren's re-enchant fee routes into the reserve. *State:* W registry
receive L130	exte	paya	–	Fee inflow. Splits across the ENCHANTED set; with no active shares the pot sweeps to the treasury instead of banking up. *State:* W totalDeposited, accPerShare, residual *Calls:* treasury.call
pending L150	publ	view	–	Claimable ETH for a fren (0 unless enchanted by its current owner). *State:* R accPerShare, debtOf, enchantedBy *Calls:* mifrens.ownerOf
isEnchanted L158	exte	view	–	Whether a fren is currently drawing fees. *Calls:* mifrens.ownerOf
castSpell L168	exte	–	–	Enchant a fren you own so it joins the earning set. *Calls:* _castSpell
castMany L173	exte	–	–	Batch enchant. *Calls:* _castSpell
_castSpell L178	priv	–	–	Join (collect the fee, bump activeShares) or re-point a stale enchantment (settle the prior caster, count unchanged). *State:* W activeShares, debtOf, enchantedBy, owed *Calls:* mifrens.ownerOf, _collectEnchantFee
_collectEnchant Fee L205	priv	–	–	Charges MOVED frens (and every forged fren) the reserve-growing re-enchant fee; original OGs are free. *Calls:* registry.enchantFee/currentToken/ donateToReserve, IERC20.transferFrom/approve
onMiFren Transfer L224	exte	–	–	Collection-only hook: settles the leaver and frees its active share. Only ever fired for GENESIS ids (finding M-06). *State:* W owed, activeShares, enchantedBy, debtOf
claim L238	publ	–	non Reentrant	Claims one fren's accrued ETH (CEI + nonReentrant). *Calls:* _claim
claimMany L243	exte	–	non Reentrant	Batch claim. *Calls:* _claim
withdrawOwed L254	exte	–	non Reentrant	Pull-withdraw of ETH settled when a fren left the active set. *State:* W owed, totalClaimed *Calls:* msg.sender.call

Function	Vis.	Mut.	Access	Role, state touched, external calls
`_claim` L265	priv	–	–	Ownership + enchantment checks, debt update before the send. *State:* W debtOf, totalClaimed *Calls:* mifrens.ownerOf, msg.sender.call

`cauldron/MiFrensGenesis.sol` — interface `IRegistrySummon` (2 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`summon` L16	exte	paya	–	Interface declaration — Registry summon surface the presale ignites.
`summoned` L17	exte	view	–	Interface declaration — Registry summon surface the presale ignites.

`cauldron/MiFrensGenesis.sol` — interface `IMiFrensDividendHook` (1 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`onMiFren Transfer` L21	exte	–	–	Interface declaration — Spell-break ping on a genesis transfer.

`cauldron/MiFrensGenesis.sol` — contract `MiFrensGenesis` (35 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`constructor` L174	inte	–	–	Fixes the OG tranche size, art cap (below the badge id range), price and per-wallet cap. *State:* W GENESIS_SUPPLY, MAX_SUPPLY, PRICE, MAX_PER_WALLET, _base, deployer
`setRegistry` L196	exte	–	–	One-shot deployer wiring of the registry this presale ignites. *State:* W registry
`mint` L209	exte	paya	non Reentrant	VOLUME mint (the iteration-#2 continuation): hook-only, records the mint block for the reveal roll. *State:* W minted, mintBlockOf *Calls:* ERC721._mint
`cancelPresale` L236	exte	–	–	Deployer safety valve for a stalled presale; opens refunds. *State:* W cancelled
`refund` L247	exte	–	non Reentrant	Post-cancel reclaim of 100% of what a wallet paid (CEI). *State:* W paid *Calls:* msg.sender.call
`totalMinted` L262	exte	view	–	ICauldronCollection surface. *State:* R minted
`maxSupply` L267	exte	view	–	ICauldronCollection surface.
`setMinter` L280	exte	–	only Deployer Or Registry	Registry/deployer wiring of the volume hook for the iteration-#2 continuation. *State:* W minter
`setVault` L285	exte	–	only Deployer Or Registry	Registry/deployer wiring of the per-iteration floor vault. *State:* W vault
`setDividend` L290	exte	–	only Deployer Or Registry	Wires the dividend so genesis transfers break the enchantment. *State:* W dividend

Function	Vis.	Mut.	Access	Role, state touched, external calls
setLiquidator Minter L298	exte	–	–	Deployer/registry/minter wiring of the badge minter. *State:* W liquidatorMinter
setLiquidator URI L304	exte	–	–	Deployer-gated badge metadata base. *State:* W liquidatorURI
mintLiquidator L312	exte	–	–	Engine-only trophy mint in the badge id range. NOTE: this collection is ERC721Votes, so a badge carries a governance vote. *State:* W liquidatorMinted, isLiquidatoor *Calls:* ERC721._mint
liquidatoor Trait L321	exte	view	–	Marketplace helper.
custodyTransfer L336	exte	–	–	REGISTRY-gated approval-free move (the OG recycle). Correctly gated, unlike CauldronCollection's. *Calls:* ERC721._transfer
setFinalizer L345	exte	–	–	Restricts who may ignite genesis, so the team's atomic summon+buy cannot be front-run. *State:* W finalizer
setMetadata L351	exte	–	–	Deployer-gated art source config. *State:* W mode, renderer, _base
setRarityOdds L361	exte	–	–	Deployer-gated gacha odds. *State:* W rarityCumBps
setRoyalty L368	exte	–	–	Deployer-gated EIP-2981 receiver + rate. *Calls:* _setDefaultRoyalty
getTransfer Validator L377	exte	view	–	ERC-721C discovery.
getTransfer Validation Function L382	exte	pure	–	ERC-721C discovery.
setTransfer Validator L387	exte	–	–	Deployer-gated validator wiring. *State:* W transferValidator
mint L396	exte	–	–	VOLUME mint (the iteration-#2 continuation): hook-only, records the mint block for the reveal roll. *State:* W minted, mintBlockOf *Calls:* ERC721._mint
reveal L409	exte	–	–	Owner-only rarity reveal with the same 256-block fallback as CauldronCollection (finding M-03). *State:* W rarityOf, revealed
burnFromVault L424	exte	–	–	Vault-only burn on floor redemption. *Calls:* ERC721._burn
_rollRarity L429	priv	view	–	Cumulative-bps tier roll.
finalize L445	exte	–	non Reentrant	IGNITION: forwards the entire treasury into registry.summon(). One-shot, sellout-gated, optionally finalizer-gated. *State:* W finalized *Calls:* registry.summon
soldOut L464	exte	view	–	Whether the OG tranche has minted out.
remaining L469	exte	view	–	OG tranche remaining.
isGenesis L480	publ	view	–	The permanent OG mark, derived from the id and the immutable tranche size.
ogTrait L487	exte	view	–	"Genesis" or "Standard" for metadata.
tokenURI L491	publ	view	–	Badge / unrevealed / renderer / baseURI resolution. *Calls:* ICollectionRenderer.tokenURI
_update L514	inte	–	–	ERC721+ERC721Votes diamond resolution, auto-delegation, dividend spell-break and everMoved — all gated on 'tokenId <= GENESIS_SUPPLY' (finding M-06). *State:* W everMoved, voting checkpoints *Calls:* ITransferValidator.validateTransfer, IMiFrensDividendHook.onMiFrenTransfer (try/catch)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`_increaseBalance` L544	inte	–	–	ERC721Votes plumbing.
`supportsInterface` L552	publ	view	–	ERC165 over ERC721 + ERC2981 + ICreatorToken.

`cauldron/MigrationVesting.sol` — interface `IVestingRegistry` (3 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`currentGeneration` L14	exte	view	–	Interface declaration — Registry migration primitive the escrow wraps.
`generationToken` L15	exte	view	–	Interface declaration — Registry migration primitive the escrow wraps.
`claimByBurn` L16	exte	–	–	Interface declaration — Registry migration primitive the escrow wraps.

`cauldron/MigrationVesting.sol` — interface `IStakerOracle` (1 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`isInstant` L24	exte	view	–	Interface declaration — Pluggable instant-tier policy.

`cauldron/MigrationVesting.sol` — contract `MigrationVesting` (15 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`constructor` L114	inte	–	–	Binds the registry, the governance owner, the initial window (bounded 1h–14d) and the instant-tier oracle. *State:* W vestWindow, stakerOracle
`startVest` L135	exte	–	non Reentrant	Pulls the caller's dead-gen tokens, migrates 1:1 through the registry, books a linear grant, and auto-releases if the wallet is instant-tier. *State:* W _grants *Calls:* IERC20.transferFrom, registry.claimByBurn, _release
`vestBatch` L153	exte	–	non Reentrant	Keeper batch: books grants for every holder who has approved the escrow. Best-effort per holder. *State:* W _grants *Calls:* IERC20.balanceOf/allowance, _pullAndVest
`_pullAndVest` L169	priv	–	–	Pull + migrate + book, verifying the live-token delta actually landed. *State:* W _grants *Calls:* IERC20.transferFrom/balanceOf, registry.claimByBurn, _isInstant
`claim` L206	exte	–	non Reentrant	Releases everything currently vested across the caller's grants. *State:* W _grants *Calls:* _release
`claimFor` L213	exte	–	non Reentrant	Permissionless release TO the beneficiary (keeper convenience). *State:* W _grants *Calls:* _release

Function	Vis.	Mut.	Access	Role, state touched, external calls
`_release` L221	priv	–	–	Pays vested-minus-released per grant and prunes drained grants (swap-pop). Per-grant token pinning survives a mid-vest relaunch. *State:* W _grants *Calls:* IERC20.transfer
`_vestedOf` L245	priv	view	–	Linear schedule; window 0 = instant.
`_isInstant` L252	priv	view	–	Consults the staker oracle; a reverting oracle means "not instant" (the safe outcome). *Calls:* stakerOracle.isInstant (try/catch)
`claimable` L265	exte	view	–	Total claimable across a holder's grants.
`locked` L273	exte	view	–	Total still locked across a holder's grants.
`grantCount` L281	exte	view	–	Number of open grants.
`grantAt` L286	exte	view	–	Reads one grant.
`setVestWindow` L296	exte	–	only Owner	Governance-tunable window for FUTURE grants (existing grants keep their snapshot). *State:* W vestWindow
`setStakerOracle` L303	exte	–	only Owner	Swaps the instant-tier policy. *State:* W stakerOracle

`cauldron/PerpEngine.sol` — interface `IPerpRegistry` (6 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`currentToken` L23	exte	view	–	Interface declaration — Registry reads the perp engine needs.
`current Generation` L24	exte	view	–	Interface declaration — Registry reads the perp engine needs.
`lastSummonAt` L25	exte	view	–	Interface declaration — Registry reads the perp engine needs.
`generationPool Id` L26	exte	view	–	Interface declaration — Registry reads the perp engine needs.
`generationToken` L27	exte	view	–	Interface declaration — Registry reads the perp engine needs.
`claimByBurn` L28	exte	–	–	Interface declaration — Registry reads the perp engine needs.

`cauldron/PerpEngine.sol` — interface `IPerpHook` (2 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`isDead` L32	exte	view	–	Interface declaration — Hook reads the perp engine needs.
`collection` L34	exte	view	–	Interface declaration — Hook reads the perp engine needs.

`cauldron/PerpEngine.sol` — contract `PerpEngine` (85 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`constructor` L276	inte	–	–	Binds poolManager/hook/registry/mifrens, seeds the tier table, the TWAP ring and the synced generation. *State:* W dividend, treasury, nftBeneficiary, tierDepthWei, tierLeverage, lastFundingAt, observations, obsIndex, lastObsTs, lastTick, syncedGeneration, syncedToken *Calls:* registry.currentGeneration/currentToken, poolManager.getSlot0

Function	Vis.	Mut.	Access	Role, state touched, external calls
totalEth L317	publ	view	–	ETH the ETH-vault owns = free PLV + lent to open longs (insurance excluded). *State:* R plv, longOiEth
freeEth L319	exte	view	–	Instantly withdrawable ETH. *State:* R plv
totalTokenAssets L321	publ	view	–	Token the token-vault owns = free inventory + lent to shorts. *State:* R plvToken, shortOiToken
freeToken L323	exte	view	–	Instantly withdrawable token inventory. *State:* R plvToken
_key L328	inte	view	–	Rebuilds the live PoolKey from registry.currentToken() on every call. *Calls:* registry.currentToken
_sqrtP L332	inte	view	–	Spot sqrtPrice of the live pool. *Calls:* poolManager.getSlot0
_currentTick L333	inte	view	–	Spot tick of the live pool. *Calls:* poolManager.getSlot0
poke L339	exte	–	–	Permissionless keeper entry that refreshes the TWAP observation and accrues funding. *Calls:* _pokeFunding
_writeObs L341	inte	–	–	Writes a TWAP observation, throttled to OBS_INTERVAL so the ring cannot be flooded. *State:* W tickCumulative, observations, obsIndex, lastObsTs, lastTick *Calls:* _currentTick
twapTick L358	publ	view	–	Time-weighted average tick over twapWindow, falling back to the oldest observation if it still spans MIN_TWAP. *State:* R observations, tickCumulative, lastObsTs, lastTick, twapWindow
markSqrtPriceX96 L384	publ	view	–	The manipulation-resistant liquidation mark (TWAP tick; spot only at genuine cold start). *Calls:* twapTick, _sqrtP
activeEthDepth L389	publ	view	–	ETH-equivalent depth at the current tick; drives leverage tiers, notional caps and the per-block liquidation cap. *Calls:* poolManager.getLiquidity/getSlot0
maxLeverage L399	publ	view	–	Depth-tiered leverage, clamped by maxLeverageCeiling. *State:* R tierDepthWei, tierLeverage, maxLeverageCeiling *Calls:* activeEthDepth
_quoteAt L412	inte	pure	–	token → ETH at a given sqrtPrice.
_quoteEth L416	inte	view	–	token → ETH at SPOT (funding sizing — spot-manipulable, see finding L-05). *Calls:* _sqrtP
_quoteMark L418	inte	view	–	token → ETH at the TWAP MARK (liquidation trigger). *Calls:* markSqrtPriceX96
_pokeFunding L421	inte	–	–	Accrues the global funding index from the signed OI imbalance × elapsed time. *State:* W lastFundingAt, fundingIndex *Calls:* _writeObs, _quoteEth
_fundingDelta L442	inte	view	–	Signed funding P&L for one position, notional pinned to entry and bounded to ±maxFundingBps of collateral. *State:* R fundingIndex, maxFundingBps
fundingDelta L455	exte	view	–	UI view of the above.
openLong L466	exte	paya	–	Opens a LONG: takes the open fee, borrows ETH from the PLV, buys token (real price impact), books the position, then best-effort sweeps liquidations. *State:* W plv, longOiEth, positions, nextId, openCount, _openIds, _openPos *Calls:* _guardOpen, _pokeFunding, _takeFee, _swapExactIn, _book, _sweepAfterOpen

Function	Vis.	Mut.	Access	Role, state touched, external calls
`openLong` L473	publ	paya	non Reentrant, not Nested	Opens a LONG: takes the open fee, borrows ETH from the PLV, buys token (real price impact), books the position, then best-effort sweeps liquidations. *State:* W plv, longOiEth, positions, nextId, openCount, _openIds, _openPos *Calls:* _guardOpen, _pokeFunding, _takeFee, _swapExactIn, _book, _sweepAfterOpen
`openShort` L503	publ	paya	non Reentrant, not Nested	Opens a SHORT: borrows token inventory, sells it (price down), holds ETH backing. *State:* W plvToken, shortOiToken, positions, openCount, _openIds *Calls:* _guardOpen, _ethToToken, _swapExactIn, _book, _sweepAfterOpen
`close` L535	exte	–	non Reentrant, not Nested	Trader-only close with slippage protection. *Calls:* _pokeFunding, _settle
`liquidate` L542	exte	–	non Reentrant, not Nested	Permissionless keeper liquidation, gated on the TWAP mark and the per-timestamp notional cap. *State:* W liqBlock, liqEthThisBlock *Calls:* _quoteMark, _underwaterVal, activeEthDepth, _settle
`liquidateInSwap` L566	exte	–	–	Hook-only single in-swap liquidation; never reverts the triggering swap. *State:* W _liqReentry, _inLocked *Calls:* _pokeFunding, _tryLiquidate
`liquidateManyIn Swap` L583	exte	–	–	Hook-only batch in-swap liquidation, capped at MAX_LIQ_PER_SWAP. *State:* same *Calls:* _tryLiquidate
`sweep Liquidations` L609	exte	–	–	Hook-only HINT-FREE sweep fired from every swap; scans a rotating window and credits tx.origin. *Calls:* _doSweep
`selfSweep` L618	exte	–	–	Self-only post-open sweep entry so it can be wrapped in try/catch with its own unlock. *Calls:* _doSweep
`_sweepAfterOpen` L624	inte	–	–	Gas-reserved best-effort sweep after an open. *Calls:* this.selfSweep (try/catch)
`_doSweep` L634	inte	–	–	Bounded rotating-window scan (SWEEP_SCAN checks, MAX_LIQ_PER_SWAP kills). *State:* W sweepCursor, _liqReentry, _inLocked *Calls:* _tryLiquidate
`_tryLiquidate` L663	inte	–	–	Liquidates one position if genuinely underwater at the mark and within the cap; otherwise a silent no-op. *State:* W liqBlock, liqEthThisBlock *Calls:* _quoteMark, _underwaterVal, _settle
`forceCloseDead` L685	exte	–	non Reentrant, not Nested	Permissionless solvent close once the token is dead; pays the caller a keeper reward. *Calls:* _isDead, _pokeFunding, _settle
`forceCloseAll Dead` L700	exte	–	non Reentrant, not Nested	Closes the whole open set oldest-first (bounded to 96). Reverts wholesale if ANY position's payout send fails (finding H-04). *Calls:* _settle

Function	Vis.	Mut.	Access	Role, state touched, external calls
`syncGeneration` L725	exte	–	non Reentrant, not Nested	Re-arms the engine on a new generation: migrates the dead inventory 1:1, re-points plvToken, resets the TWAP ring. Requires openCount == 0. *State:* W plvToken, shortOiToken, longOiEth, observations, tickCumulative, obsIndex, lastObsTs, lastTick, syncedGeneration, syncedToken *Calls:* registry.claimByBurn (try/catch), IERC20.balanceOf
`isLiquidatable` L765	exte	view	–	View: is the position underwater at the mark? *Calls:* _underwater
`_underwater` L772	inte	view	–	Underwater test at the TWAP mark. *Calls:* _quoteMark, _underwaterVal
`_underwaterVal` L781	inte	view	–	Underwater test given a pre-computed mark value (long: value < debt + buffer; short: buy-back cost + buffer > backing). *State:* R maintenanceBps
`_settle` L793	inte	–	–	THE settlement core: unwinds the position through a real pool swap, repays/absorbs, applies funding, takes the liquidation penalty, pays keeper + trader. *State:* W positions, openCount, longOiEth/shortOiToken, plv, plvToken, insuranceEth *Calls:* _swapExactIn/_buyExactOut, _replenishPlv/_absorbPlvLoss, _routeFee, _sendEth, _awardBadge
`_run` L869	inte	–	–	Routes a swap through a fresh unlock or directly, depending on _inLocked. *Calls:* poolManager.unlock / _execSwap
`_swapExactIn` L874	inte	–	–	Exact-input swap helper. *Calls:* _run
`_buyExactOut` L879	inte	–	–	Exact-output buy (makes the short inventory whole). *Calls:* _run
`unlockCallback` L883	exte	–	–	PoolManager-only unlock entry. *Calls:* _execSwap
`_execSwap` L891	inte	–	–	Swap body: performs the pool swap tagged with nftBeneficiary and settles both legs. *Calls:* poolManager.swap, _settleCur, _take
`_guardOpen` L921	inte	view	–	Blocks opens before the warmup, once the token is dead, or above the depth-tiered leverage. *Calls:* registry.lastSummonAt, _isDead, maxLeverage
`_isDead` L926	inte	view	–	Reads the hook's death verdict, defaulting to "alive" on revert. *Calls:* IPerpHook.isDead (try/catch)
`_takeFee` L929	inte	–	–	Charges the 6.9% open fee (halved for fren holders) and routes it. *Calls:* mifrens.balanceOf, _routeFee
`_checkNotional` L935	inte	view	–	Caps a single position's notional at maxNotionalBps of depth (bounded slippage). *Calls:* activeEthDepth
`_book` L938	inte	–	–	Writes the position and adds it to the enumerable open set. *State:* W nextId, openCount, positions, _openIds, _openPos
`_removeOpen` L949	inte	–	–	Swap-and-pop removal from the open set. *State:* W _openIds, _openPos
`_ethToToken` L959	inte	view	–	ETH → token at spot. *Calls:* _sqrtP

Function	Vis.	Mut.	Access	Role, state touched, external calls
`_routeFee` L964	inte	–	–	Carves LP yield + insurance (side-attributed) then splits the remainder dividend/treasury. Uses a REVERTING send (finding H-04). *State:* W plv, tokYieldEth, tokYieldCumulative, insuranceEth *Calls:* _sendEth
`_sendEth` L993	inte	–	–	Raw ETH send that REVERTS on failure — the griefing primitive behind finding H-04. *Calls:* to.call
`_replenishPlv` L999	inte	–	–	Long bad debt: refills plv from insurance up to the buffer. *State:* W insuranceEth, plv
`_absorbPlvLoss` L1008	inte	–	–	Short bad debt: absorbs the overspend from insurance then LP principal (saturating). *State:* W insuranceEth, plv
`_awardBadge` L1029	inte	–	–	HYBRID trophy: gas-bounded in-swap mint, else a claimable credit. A zero collection address returns success and silently loses the badge (finding L-06). *State:* W badgesOwed *Calls:* IPerpHook.collection, ILiquidatorMintable. mintLiquidator
`claimLiquidator Badges` L1054	exte	–	non Reentrant	Claims owed badges into the LIVE collection, bounded by n. *State:* W badgesOwed *Calls:* IPerpHook.collection, mintLiquidator
`_settleCur` L1063	priv	–	–	Settles a currency leg (native via value, ERC20 via sync+transfer+settle). *Calls:* poolManager.sync/settle, _safeTransfer
`_take` L1067	priv	–	–	poolManager.take wrapper. *Calls:* poolManager.take
`_safeTransfer` L1068	priv	–	–	Return-value-tolerant ERC20 transfer. *Calls:* token.call
`fundPlv` L1079	exte	paya	not Nested, only Owner	Owner-only, SHARE-LESS permanent ETH donation to the PLV. *State:* W plv
`fundPlvToken` L1083	exte	–	not Nested, only Owner	Owner-only, share-less token inventory donation. *State:* W plvToken *Calls:* IERC20.transferFrom
`fundInsurance` L1088	exte	paya	–	Permissionless insurance top-up. *State:* W insuranceEth
`creditPerpFee` L1095	exte	paya	–	Hook-only: credits a redirected perp BUY fee into the ETH PLV. *State:* W plv
`creditPerpFee Token` L1100	exte	paya	–	Hook-only: credits a redirected perp SELL fee into the segregated token-side pot. *State:* W tokYieldEth, tokYieldCumulative
`_creditPerp` L1102	priv	–	–	Shared body; hook-gated, pure accounting so it is safe to nest mid-swap. *State:* W plv / tokYieldEth
`fundFromVault` L1115	exte	paya	only Vault	Vault-only ETH deposit into the PLV. *State:* W plv
`withdrawPlvTo` L1119	exte	–	non Reentrant, not Nested, only Vault	Vault-only pull of FREE ETH. *State:* W plv *Calls:* _sendEth

Function	Vis.	Mut.	Access	Role, state touched, external calls
withdrawTokYieldTo L1125	exte	–	non Reentrant, not Nested, only Vault	Vault-only pull from the segregated short-side yield pot. *State:* W tokYieldEth *Calls:* _sendEth
fundTokenFromVault L1130	exte	–	only Vault	Vault-only token deposit into the short inventory. *State:* W plvToken *Calls:* IERC20.transferFrom
withdrawPlvTokenTo L1136	exte	–	non Reentrant, not Nested, only Vault	Vault-only pull of FREE token inventory. *State:* W plvToken *Calls:* _safeTransfer
setFees L1143	exte	–	only Owner	Open fee / OG discount / liq penalty / dividend share / keeper cut, all bounded. *State:* W openFeeBps, ogDiscountBps, liqPenaltyBps, divShareBps, keeperBps
setRisk L1147	exte	–	only Owner	Warmup, leverage ceiling, maintenance, notional and OI caps, funding rate. Enforces warmup ≥ MIN_TWAP. *State:* W warmup, maxLeverageCeiling, maintenanceBps, maxNotionalBps, maxOiBps, fundingRateBpsPerDay
setTwapWindow L1164	exte	–	only Owner	Tunes the liquidation-mark averaging window down to a 1s floor. *State:* W twapWindow
setTiers L1168	exte	–	only Owner	Depth→leverage tier table; requires levs.length == depths.length + 1. *State:* W tierDepthWei, tierLeverage
setRouting L1171	exte	–	only Owner	Dividend + treasury sinks. *State:* W dividend, treasury
setNftBeneficiary L1173	exte	–	only Owner	Where perp-volume gacha creatures accrue. *State:* W nftBeneficiary
setGuards L1175	exte	–	only Owner	TWAP window, per-block liquidation cap, funding cap. *State:* W twapWindow, maxLiqBps, maxFundingBps
setVault L1187	exte	–	only Owner	Wires the community PLV. DRAIN GUARD: cannot be re-pointed while depositor funds are staked. *State:* W vault
setVaultSplit L1193	exte	–	only Owner	LP-yield / insurance shares of routed fees. *State:* W vaultYieldBps, insuranceBps
setVaultLimits L1199	exte	–	only Owner	Utilisation cap + insurance circuit-breaker floor. *State:* W maxUtilBps, insuranceFloor
setMinCollateral L1204	exte	–	only Owner	Dust filter on opens, capped at 1 ETH. *State:* W minCollateral
skimInsurance L1213	exte	–	only Owner	Owner may skim insurance ABOVE insuranceFloor — which defaults to 0 (finding M-05). *State:* W insuranceEth *Calls:* _sendEth
positionHealth L1229	exte	view	–	UI view: mark value, debt/backing, liquidatable. *Calls:* _quoteMark, _underwater
receive L1240	exte	paya	–	Accepts ETH (pool takes, keeper refunds).

cauldron/PerpStakerOracle.sol — interface IPerpShares (2 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
ethShareOf L8	exte	view	–	Interface declaration — PerpVault share reads.

Function	Vis.	Mut.	Access	Role, state touched, external calls
`tokShareOf` L9	exte	view	–	Interface declaration — PerpVault share reads.

`cauldron/PerpStakerOracle.sol` — contract `PerpStakerOracle` (2 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`constructor` L24	inte	–	–	Binds the PerpVault whose shares define the instant tier.
`isInstant` L29	exte	view	–	A wallet is instant-tier iff it holds a live ETH- or token-side PLV share. *Calls:* perpVault.ethShareOf/tokShareOf

`cauldron/PerpVault.sol` — interface `IPerpEngineVault` (10 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`fundFromVault` L9	exte	paya	–	Interface declaration — Engine surface the community vault drives.
`withdrawPlvTo` L10	exte	–	–	Interface declaration — Engine surface the community vault drives.
`fundTokenFrom Vault` L11	exte	–	–	Interface declaration — Engine surface the community vault drives.
`withdrawPlvToken To` L12	exte	–	–	Interface declaration — Engine surface the community vault drives.
`totalEth` L13	exte	view	–	Interface declaration — Engine surface the community vault drives.
`freeEth` L14	exte	view	–	Interface declaration — Engine surface the community vault drives.
`totalToken Assets` L15	exte	view	–	Interface declaration — Engine surface the community vault drives.
`freeToken` L16	exte	view	–	Interface declaration — Engine surface the community vault drives.
`tokYield Cumulative` L18	exte	view	–	Interface declaration — Engine surface the community vault drives.
`withdrawTokYield To` L19	exte	–	–	Interface declaration — Engine surface the community vault drives.

`cauldron/PerpVault.sol` — interface `IVaultRegistry` (1 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`currentToken` L23	exte	view	–	Interface declaration — Live-token read for the vault.

`cauldron/PerpVault.sol` — contract `PerpVault` (18 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`constructor` L103	inte	–	–	Binds the engine and the registry (for currentToken). *State:* W engine, registry
`assetsEth` L110	publ	view	–	ETH backing LIVE shares = engine.totalEth() − queued exits. *State:* R pendingEth *Calls:* engine.totalEth

Function	Vis.	Mut.	Access	Role, state touched, external calls
`assetsTok` L115	publ	view	–	Token backing live shares = engine.totalTokenAssets() − queued exits. *State:* R pendingTok *Calls:* engine.totalTokenAssets
`depositEth` L124	exte	paya	non Reentrant	Mints ETH-side shares at the pre-deposit price with a 10^6 virtual-share offset, then funds the engine. *State:* W ethShares, ethShareOf *Calls:* engine.fundFromVault
`withdrawEth` L137	exte	–	non Reentrant	Burns shares, pays up to the engine's free ETH, queues the remainder (CEI). *State:* W ethShareOf, ethShares, pendingEth, pendingEthOf *Calls:* engine.freeEth/withdrawPlvTo
`claimPendingEth` L161	exte	–	non Reentrant	Claims a queued ETH exit as liquidity frees up. *State:* W pendingEthOf, pendingEth *Calls:* engine.freeEth/withdrawPlvTo
`_syncTokYield` L177	inte	–	–	Folds newly-accrued short-side yield into the per-share accumulator. SKIPS the fold at zero shares WITHOUT advancing the watermark (finding H-05). *State:* W accEthPerTokShare, lastTokYieldCum *Calls:* engine.tokYieldCumulative
`_settleTok` L185	inte	–	–	Banks a user's earned-so-far ETH before their share count changes. *State:* W tokRewardOwed
`_resetTokDebt` L193	inte	–	–	Rebases a user's reward baseline to their current share count. *State:* W tokRewardDebt
`depositToken` L201	exte	–	non Reentrant	Stakes the live token to back shorts; pulls from the user and pushes into the engine. *State:* W tokShares, tokShareOf, tokRewardDebt *Calls:* registry.currentToken, _pull, _approve, engine.fundTokenFromVault
`claimTokYield` L219	exte	–	non Reentrant	Claims accrued short-side ETH reward from the segregated pot. *State:* W tokRewardOwed *Calls:* engine.withdrawTokYieldTo
`withdrawToken` L230	exte	–	non Reentrant	Burns token shares, pays up to free inventory, queues the rest. *State:* W tokShareOf, tokShares, pendingTok, pendingTokOf *Calls:* engine.freeToken/withdrawPlvTokenTo
`claimPendingToken` L253	exte	–	non Reentrant	Claims a queued token exit. *State:* W pendingTokOf, pendingTok *Calls:* engine.freeToken/withdrawPlvTokenTo
`ethPosition` L269	exte	view	–	UI view: redeemable / instantly available / queued ETH. *Calls:* engine.freeEth
`tokenPosition` L277	exte	view	–	UI view for the token side. *Calls:* engine.freeToken
`pendingTokYield` L286	exte	view	–	UI view of accrued-but-unclaimed short-side ETH. *Calls:* engine.tokYieldCumulative
`_pull` L297	priv	–	–	Return-value-tolerant transferFrom. *Calls:* token.call
`_approve` L303	priv	–	–	Return-value-tolerant approve. *Calls:* token.call

`cauldron/PoolOps.sol` — interface `IPositionManagerOps` (3 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`modifyLiquidities` L32	exte	paya	–	Interface declaration — Minimal PositionManager surface used by PoolOps.
`nextTokenId` L33	exte	view	–	Interface declaration — Minimal PositionManager surface used by PoolOps.
`getPositionLiquidity` L34	exte	view	–	Interface declaration — Minimal PositionManager surface used by PoolOps.

`cauldron/PoolOps.sol` — interface `ILedgerOps` (7 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`redeem` L42	exte	–	–	Interface declaration — Ledger surface PoolOps drives.
`buyback` L43	exte	–	–	Interface declaration — Ledger surface PoolOps drives.
`floorPerNFT` L44	exte	view	–	Interface declaration — Ledger surface PoolOps drives.
`credit` L45	exte	–	–	Interface declaration — Ledger surface PoolOps drives.
`crystallize` L46	exte	–	–	Interface declaration — Ledger surface PoolOps drives.
`crystallized` L47	exte	view	–	Interface declaration — Ledger surface PoolOps drives.
`totalEntitled` L48	exte	view	–	Interface declaration — Ledger surface PoolOps drives.

`cauldron/PoolOps.sol` — interface `IColMinted` (1 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`totalMinted` L52	exte	view	–	Interface declaration — Live mint-count read.

`cauldron/PoolOps.sol` — interface `IVaultRedeemedOps` (2 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`redeemed` L55	exte	view	–	Interface declaration — Vault outstanding/redeemed reads used at crystallisation.
`outstanding` L57	exte	view	–	Interface declaration — Vault outstanding/redeemed reads used at crystallisation.

`cauldron/PoolOps.sol` — interface `ICollectionOps` (2 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`custodyTransfer` L61	exte	–	–	Interface declaration — Collection custody surface.
`ownerOf` L62	exte	view	–	Interface declaration — Collection custody surface.

`cauldron/PoolOps.sol` — interface `ILegacyHookOps` (2 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`sweepLegacyReserve` L66	exte	–	–	Interface declaration — Hook legacy-sweep surface.
`legacyRegistry` L67	exte	view	–	Interface declaration — Hook legacy-sweep surface.

`cauldron/PoolOps.sol` — interface `ICauldronBurn` (1 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`burn` L71	exte	–	–	Interface declaration — Token burn surface.

`cauldron/PoolOps.sol` — interface `IAutoFlag` (1 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`autoMigrate` L74	exte	view	–	Interface declaration — Auto-migrate opt-in read (self-call under delegatecall).

`cauldron/PoolOps.sol` — interface `IPermit2Ops` (1 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`approve` L87	exte	–	–	Interface declaration — Permit2 approve surface.

`cauldron/PoolOps.sol` — library `PoolOps` (21 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`creatureFor` L139	exte	pure	–	The cycling gen-1/fallback creature name+symbol table (kept out of the registry for EIP-170).
`_approve` L157	priv	–	–	ERC20 approve to Permit2 + Permit2 approve to the PositionManager (300s expiry). *Calls:* IERC20.approve, IPermit2Ops.approve
`_sqrtPrice` L162	priv	pure	–	Babylonian sqrt of (tokenAmount ≪ 192)/ethAmount. Reverts on ethAmount == 0.
`createAndSeed` L178	exte	–	–	Reference two-position seed (no buy): init pool, mint the full-range active position, mint the out-of-range reserve. *Calls:* poolManager.initialize, _seedActive, _seedReserve
`createAndSeed Progressive` L230	exte	–	–	PROGRESSIVE handoff: places ledger B (reserve) in full, approves ledger A to the seeder and starts the campaign. activePositionId stays 0. *Calls:* poolManager.initialize, _seedReserve, IERC20. approve, ISeeder.startSeed
`createAndSeedWith Buy` L302	exte	–	–	GREEN CANDLE: seed 100% of supply at a discount, buy exactly the reserve back out (a real first-block trade), re-park it out of range below the post-buy spot. *Calls:* poolManager.initialize/unlock/getSlot0, _seedActive, _seedReserve
`executeBuy` L361	exte	–	–	Unlock body for the seed/prime buy, running in the REGISTRY's context; tags hookData with the registry so the hook waives the fee. *Calls:* poolManager.swap/settle/take
`primeBuy` L399	exte	–	–	Owner-funded exact-input market buy whose output goes straight to the treasury (net demand, zero dilution). *Calls:* poolManager.unlock
`deployToken` L415	exte	–	–	CREATE2 token deploy (salt = generation) with address(this) = the registry. *Calls:* new CauldronToken
`_seedActive` L423	priv	–	–	Mints the full-range ACTIVE position (ETH + tradeable tokens) and sweeps excess ETH back. *Calls:* _approve, pm.nextTokenId, pm.modifyLiquidities
`_seedReserve` L462	priv	–	–	Mints the single-sided RESERVE position below the launch tick (pure token1). *Calls:* ReserveLib.liquidityForTokenOut, _approve, pm. modifyLiquidities

Function	Vis.	Mut.	Access	Role, state touched, external calls
`removeAll` L494	exte	–	–	Removes 100% of a position, takes both currencies, burns the NFT; returns balance deltas. *Calls:* pm.getPositionLiquidity, pm.modifyLiquidities
`claimFrom Reserve` L524	publ	–	–	Pulls EXACTLY 'amount' from the reserve to a recipient — but CAPS at the position's liquidity and returns silently short (finding H-03). *Calls:* ReserveLib.liquidityForTokenOut, pm. getPositionLiquidity, pm.modifyLiquidities
`addToReserve` L564	publ	–	–	Single-sided top-up of the reserve position; returns the amount actually consumed. *Calls:* _approve, pm.modifyLiquidities
`migrateOne` L601	publ	–	–	Burn old + claim the same from the reserve. Shared by claimByBurn and the keeper batch. *Calls:* ICauldronBurn.burn, claimFromReserve
`autoMigrate Batch` L611	exte	–	–	Keeper loop over opted-in holders; reads the opt-in flag via a self-call so it resolves to the registry. *Calls:* IAutoFlag.autoMigrate, IERC20.balanceOf, migrateOne
`doLegacyNote` L630	publ	–	–	Splits a live buyback between the forged tranche's ledger and the OG genesis reserve when iteration #2 continues MiFrens. *Calls:* IColMinted.totalMinted, ILedgerOps.credit
`materialize Legacy` L657	exte	–	–	Sweeps the hook's buyback tokens and either deposits them into the reserve (live) or burns them and carries the value as a number (relaunch flush). *Calls:* ILegacyHookOps.legacyRegistry/ sweepLegacyReserve, addToReserve, ICauldronBurn.burn, doLegacyNote
`crystallize Collection` L682	exte	–	–	At death, freezes a collection's supply and folds the final swept-ETH sizing into its token entitlement. *Calls:* ILedgerOps.crystallized/crystallize, IVaultRedeemedOps.outstanding
`recycle Collection` L703	exte	–	–	Debits the ledger, moves the NFT to the treasury, pays the floor from the live reserve. Calls collection.custodyTransfer — which the factory-deployed collection rejects (finding H-01). *Calls:* ICollectionOps.ownerOf/custodyTransfer, ILedgerOps.redeem, claimFromReserve
`buyCollection` L723	exte	–	–	Buys a treasury NFT at 2× floor, adds the payment to the reserve, ratchets the ledger, hands the NFT over. *Calls:* ILedgerOps.floorPerNFT/buyback, IERC20. transferFrom, addToReserve, custodyTransfer

`cauldron/RedemptionExt.sol` — **contract** `RedemptionExt` (5 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`redeemOgFren` L54	exte	–	non Reentrant	RECYCLE an OG fren for its live floor share. Effects first (debit reserve, move NFT to treasury), then pull tokens from the reserve LP. *State:* W genesisReserveOutstanding (registry's) *Calls:* IERC721.ownerOf, IMiFrensContinuable. custodyTransfer, PoolOps.claimFromReserve
`buyTreasuryOg Fren` L86	exte	–	non Reentrant	Buys a treasury-held fren for 2× floor, paid in the live token, which is added to the reserve (floor ratchet). *State:* W genesisReserveOutstanding *Calls:* IERC721.ownerOf, _pullGrow, custodyTransfer

Function	Vis.	Mut.	Access	Role, state touched, external calls
`donateToReserve` L107	exte	–	non Reentrant	Permissionless floor growth; the re-enchant fee routes through here. *State:* W genesisReserveOutstanding *Calls:* _pullGrow
`materializeLegacy Reserve` L118	exte	–	non Reentrant	Permissionless: sweeps the hook's held buyback tokens INTO the reserve and credits the ledger in one step (Invariant R by construction). *State:* W genesisPending *Calls:* PoolOps.materializeLegacy
`_pullGrow` L136	priv	–	–	transferFrom + addToReserve + reserve accounting; emits FloorGrew. *State:* W genesisReserveOutstanding *Calls:* IERC20.transferFrom, PoolOps.addToReserve

`cauldron/ReserveLib.sol` — library `ReserveLib` (4 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`_alignDown` L26	inte	pure	–	Aligns a tick DOWN to a spacing multiple (toward $-\infty$).
`_alignUp` L33	inte	pure	–	Aligns a tick UP to a spacing multiple (toward $+\infty$).
`reserveTicks` L49	inte	pure	–	The reserve band: from the lowest usable aligned tick up to launchTick − offset, collapsing to one spacing if degenerate. ORIENTATION is load-bearing (below launch ⇒ pure token1).
`liquidityForToken Out` L74	inte	pure	–	Liquidity representing exactly 'amount1' for a fully-below-price range, rounding DOWN. Reverts SafeCastOverflow for pathologically low ticks (finding I-05). *Calls:* LiquidityAmounts.getLiquidityForAmount1

`cauldron/RoyaltyRouter.sol` — interface `ILegacyBuffer` (1 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`fundLegacyBuffer` L5	exte	paya	–	Interface declaration — Hook buffer top-up surface.

`cauldron/RoyaltyRouter.sol` — contract `RoyaltyRouter` (2 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`constructor` L27	inte	–	–	Binds the hook whose buyback buffer these royalties fund.
`receive` L32	exte	paya	–	Forwards marketplace royalty ETH straight into the hook's legacy buyback buffer. *Calls:* hook.fundLegacyBuffer

`cauldron/SeedLib.sol` — library `SeedLib` (7 functions)

Function	Vis.	Mut.	Access	Role, state touched, external calls
`_alignDown` L40	inte	pure	–	Tick alignment (down).
`_alignUp` L45	inte	pure	–	Tick alignment (up).
`deployedTarget Wad` L58	inte	pure	–	The progressive schedule: linear from seedFloor at start to 100% at start+window; window 0 = atomic. Pure function of elapsed time, hence un-accelerable.

Function	Vis.	Mut.	Access	Role, state touched, external calls
`askBand` L84	inte	pure	–	The i-th single-sided TOKEN band BELOW the launch tick (what buyers eat).
`bidBand` L111	inte	pure	–	The j-th single-sided ETH band ABOVE the launch tick (what sellers hit).
`_bandWidth` L128	inte	pure	–	Equal band width tiling an offset into n bands, never below one spacing.
`taperWeightWad` L140	inte	pure	–	Descending per-band weight, summing to 1e18 bar dust.